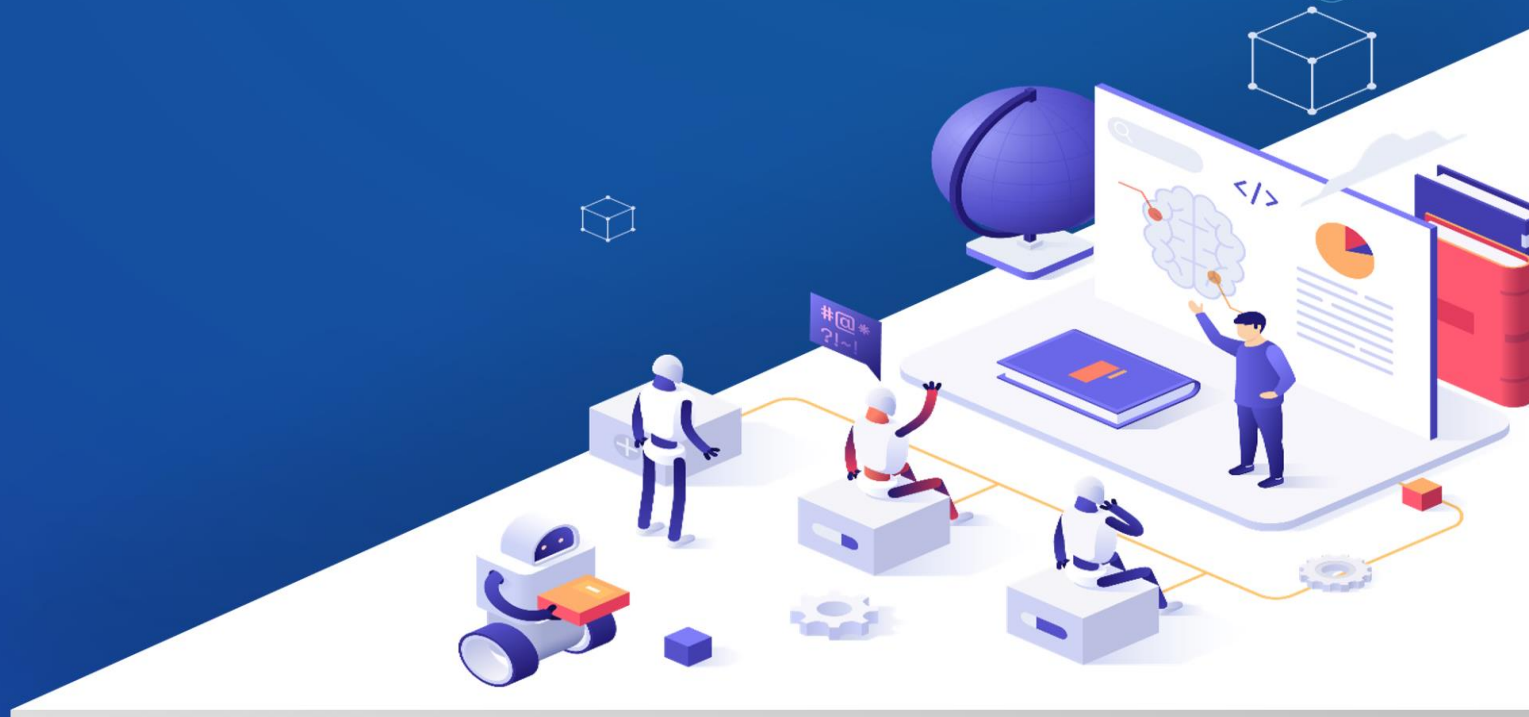
RAG(Retrieval Augmented Generation) 소개 및 실습

Colab 기반 실습 - 기존 GPU 환경의 소스파일은 동일하며 실행 환경이 colab

배 경 만(kyoungman.bae@etri.re.kr)
시각지능연구실 / 한국전자통신연구원



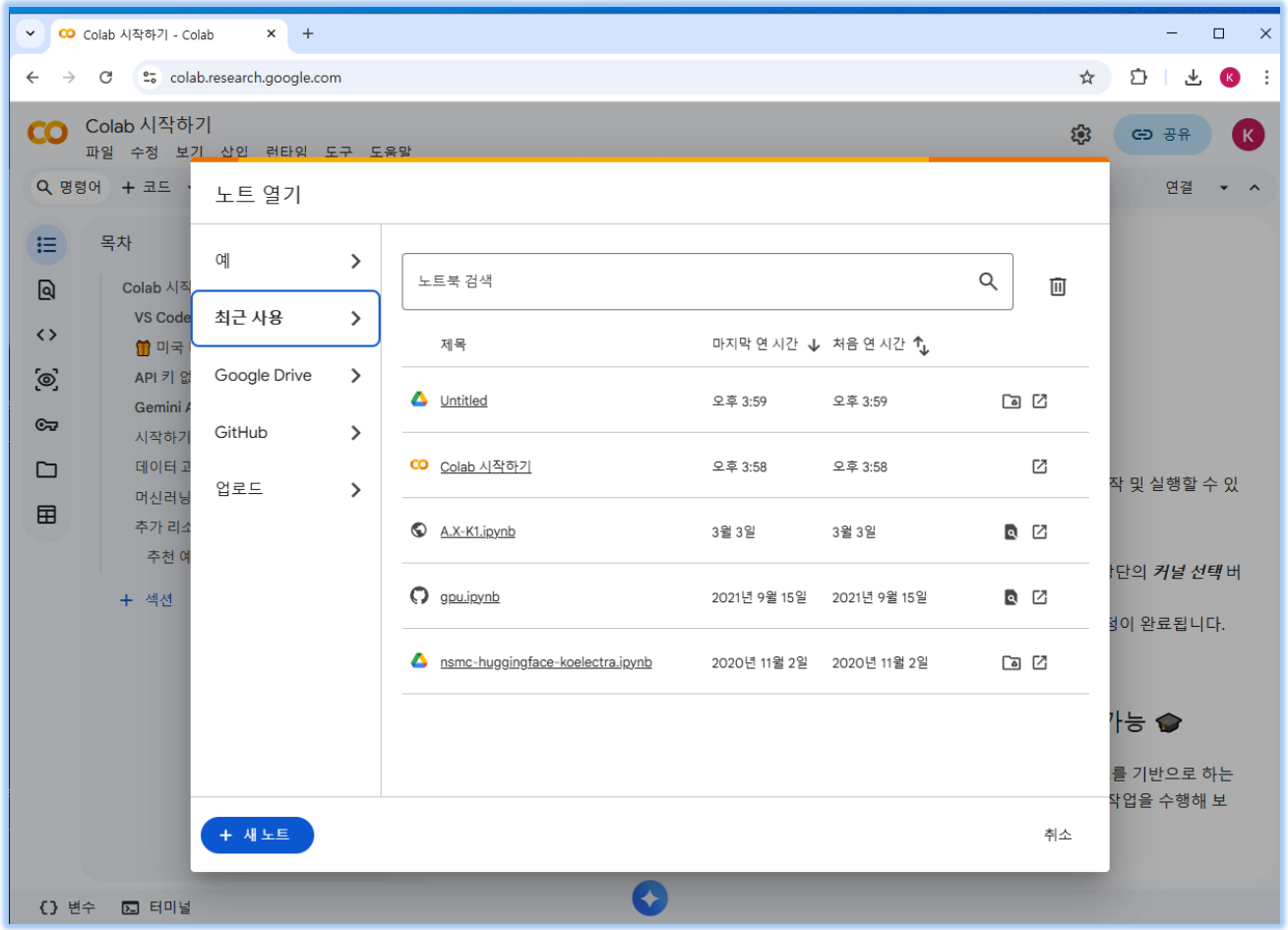
Chapter

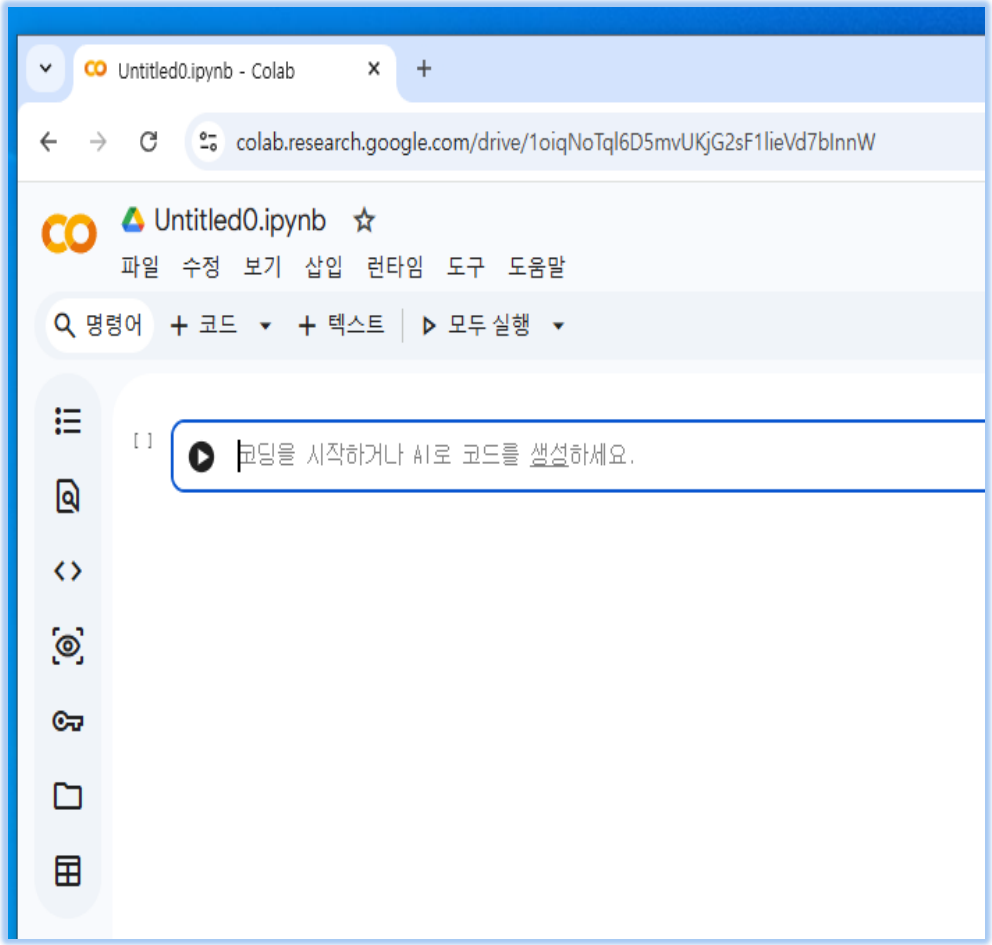
1

환경 설정

CoLab

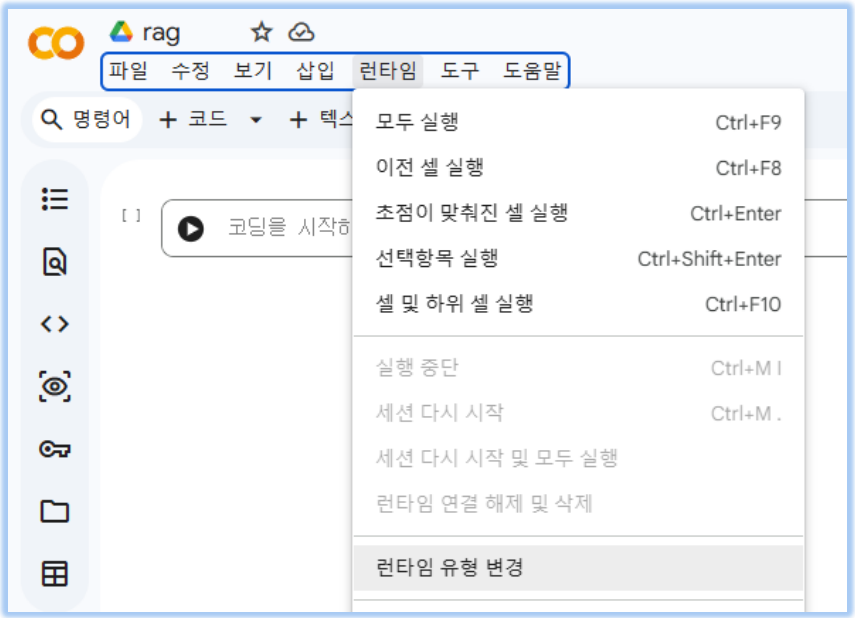
- 크롬이나 엣지 실행
  - 구글 계정 로그인 후 colab 열기(colab.research.google.com)
  - 새 노트 클릭 시 파이썬 실행 가능

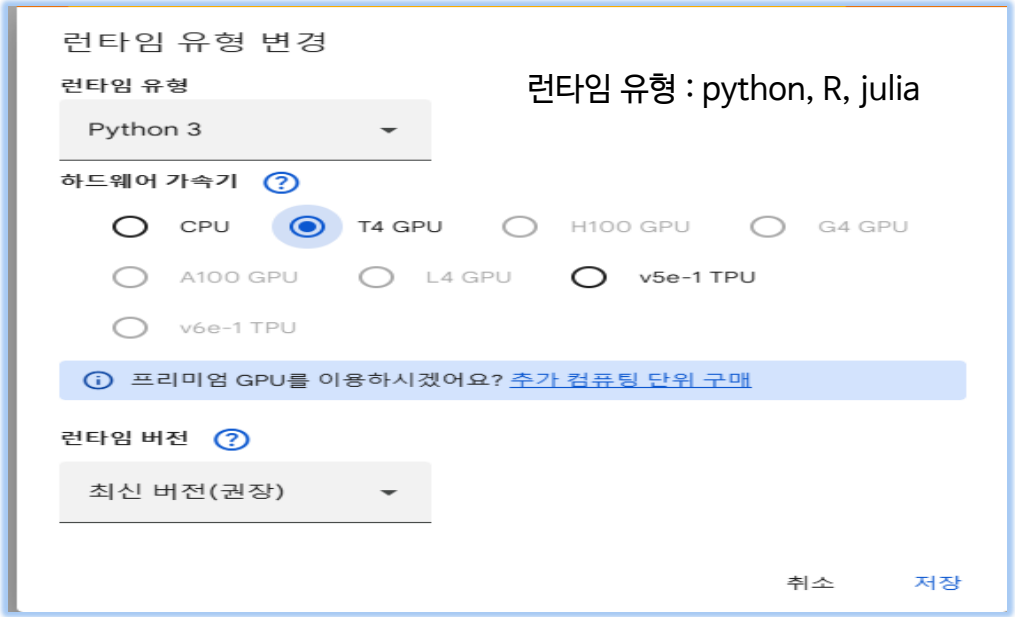




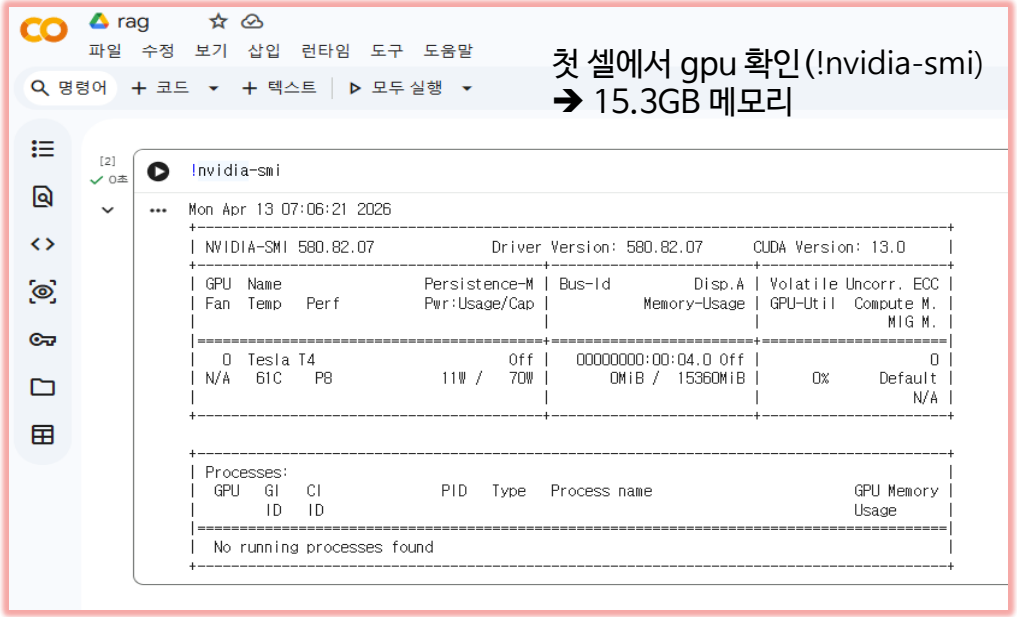
Copyright© 2021 by ETRI

- GPU 런타임 설정





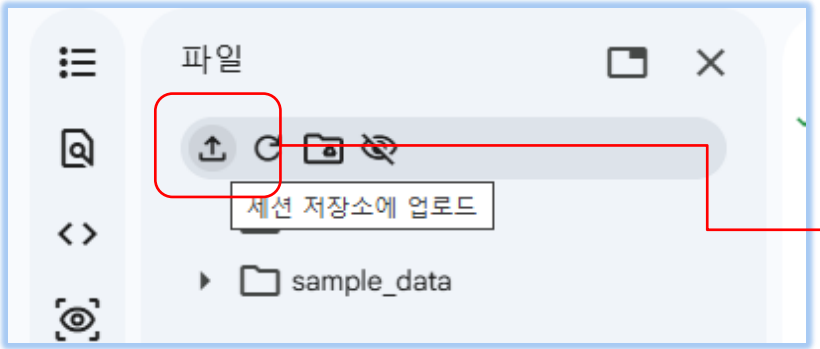
런타임 유형 : python, R, julia



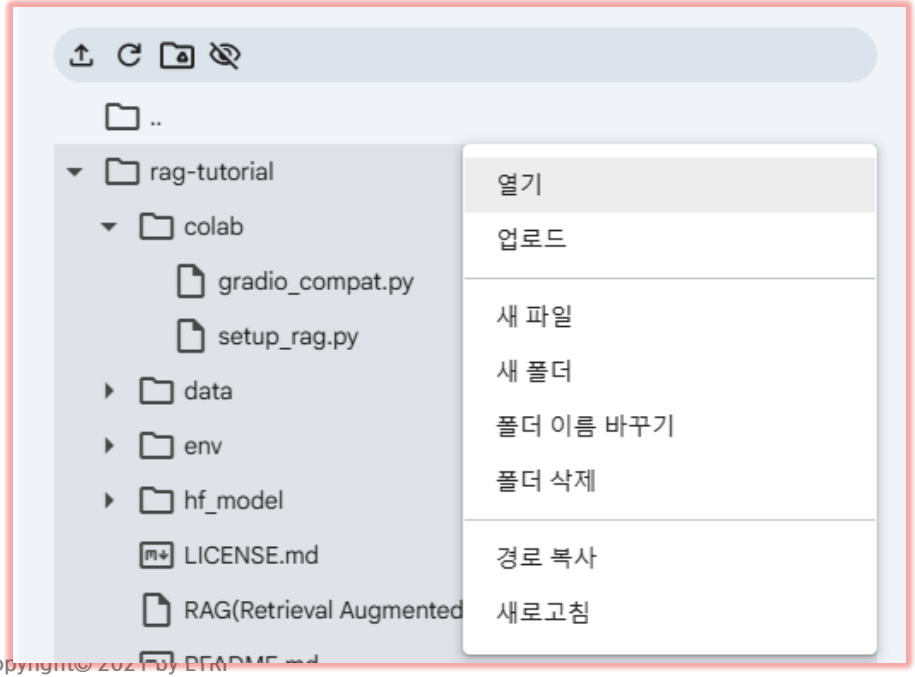
첫 셀에서 gpu 확인(!nvidia-smi)
➔ 15.3GB 메모리

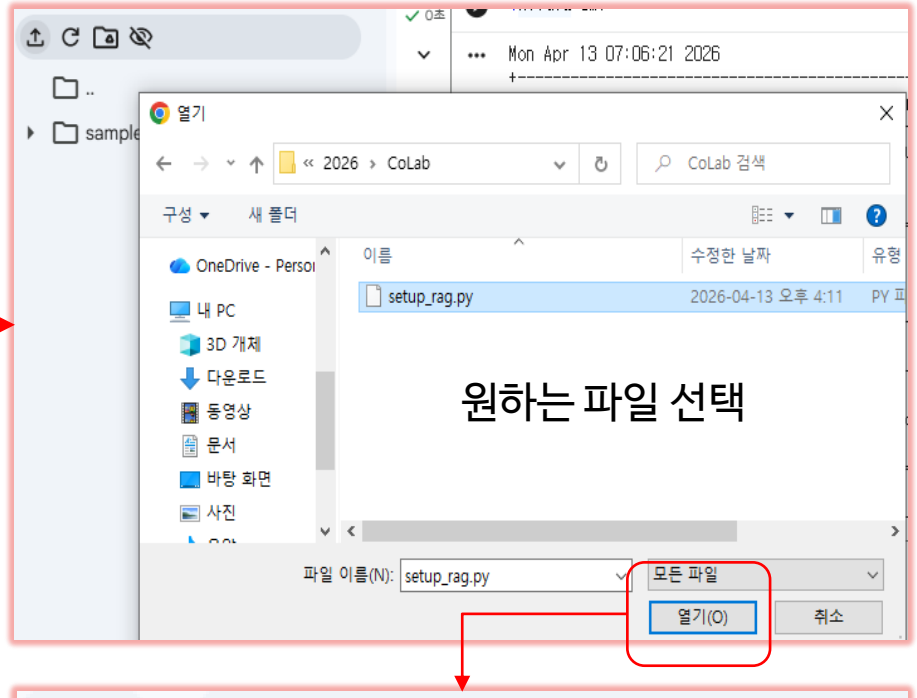
Copyright© 2021 by ETRI

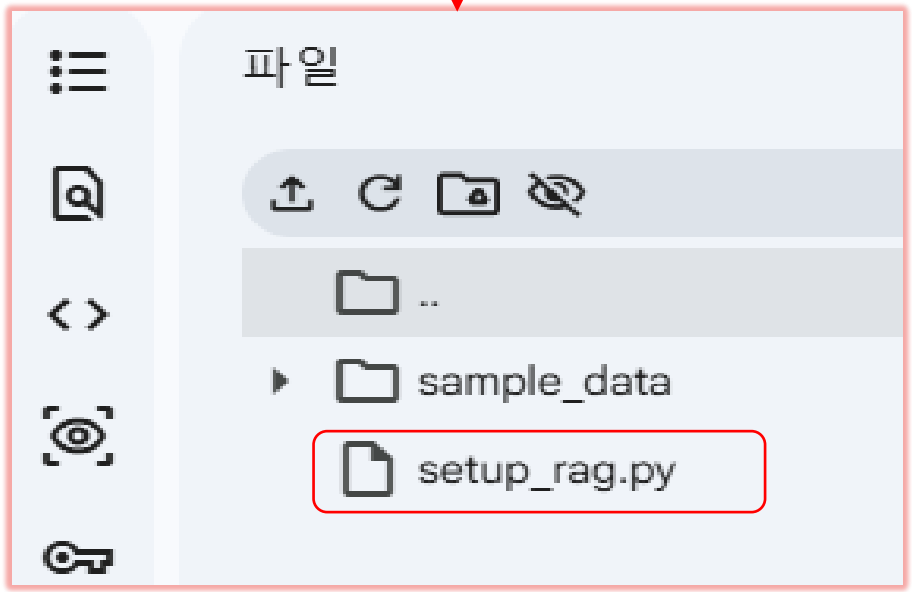
- 파일 업로드



폴더별 오른쪽 끝 ⋮ 클릭시 아래와 같은 서브 메뉴 실행가능



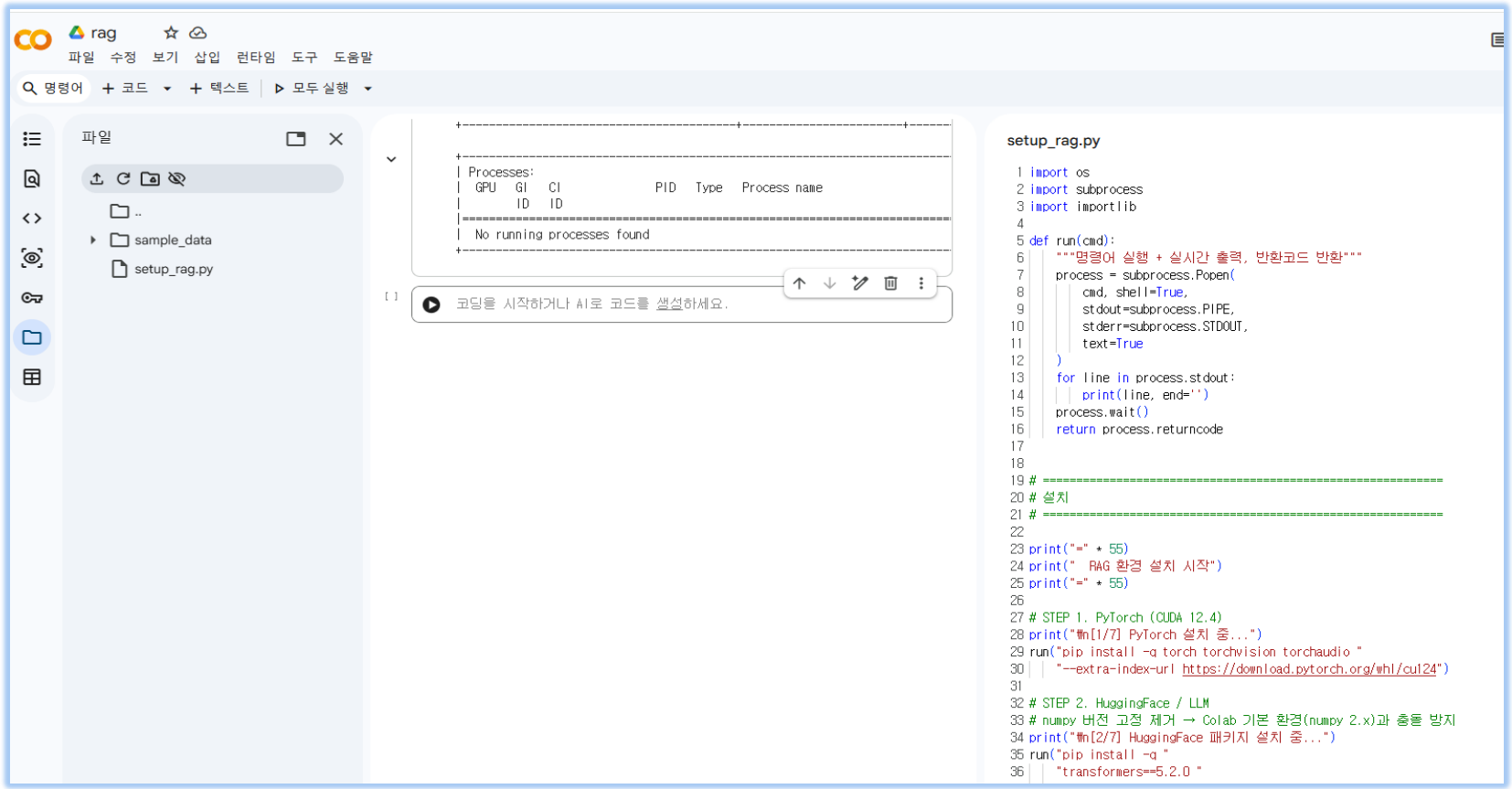




업로드 확인

Copyright© 2021 by ETRI

- 파일 수정
  - 원하는 파일 더블클릭 ➔ 셀 오른쪽에 에디터 탭에서 수정 ➔ crtl+s
  - 셀에서 sed 명령어로 특정 문자열 찾아서 수정
    - !sed -i 's/duckduckgo_search/ddgs/g' setup_rag.py



Copyright© 2021 by ETRI

- 기본 명령어
  - Shift+enter : 현재 셀의 명령어 실행
  - %cd ➔ 폴더 경로 이동. 기존 명령어 앞에 %가 들어감
  - !python : 기존 명령어에 !가 들어가야 실행이 됨
  - b: 현재셀 아래에 새로운 셀을 추가

```
%cd ..
/content

!ls
rag-tutorial  sample_data

%cd rag
rag-tutorial/
range
RuntimeWarning
ResourceWarning
```

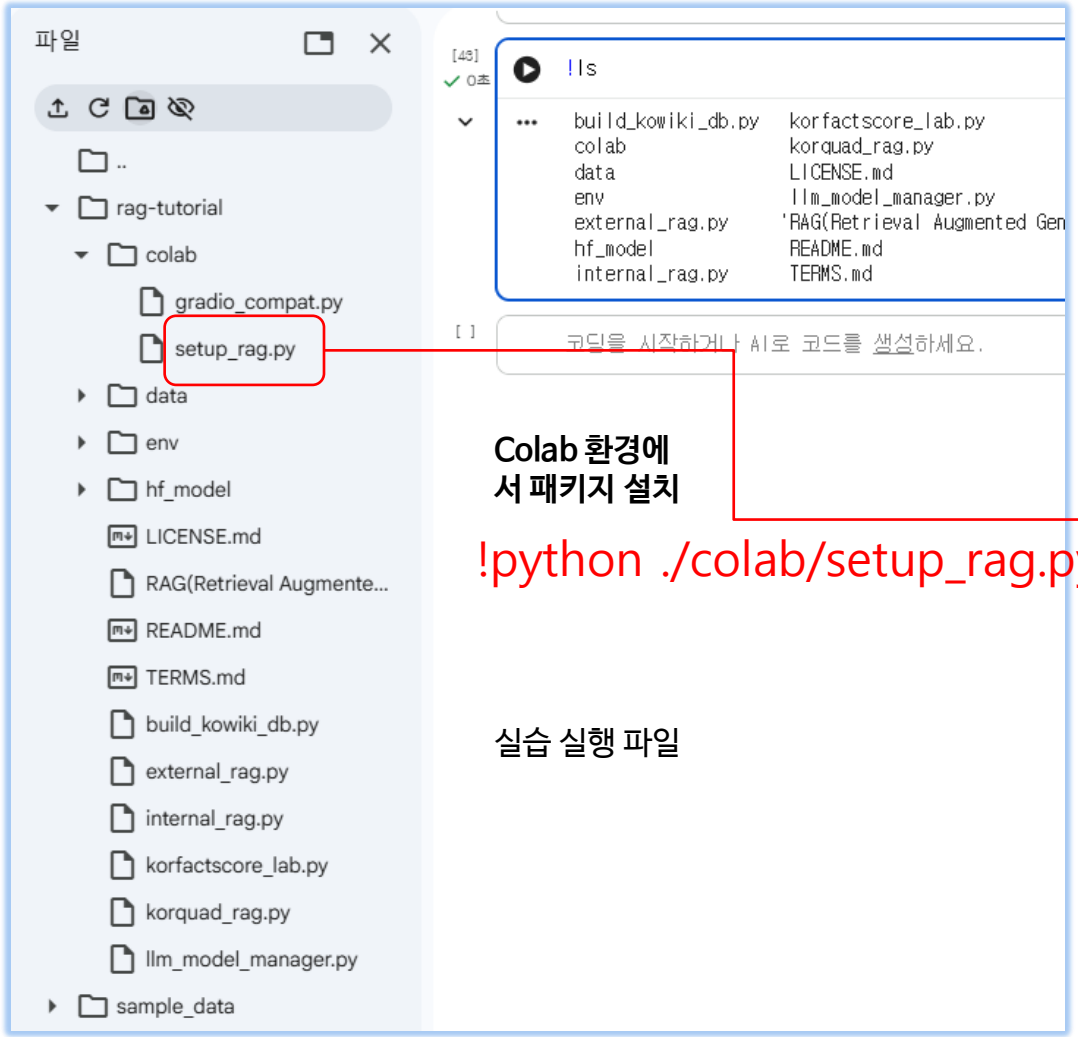
현재 폴더 기준 자동 완성

이동한 경로 확인(현재 경로 확인)

```
!pwd
/content/rag-tutorial

!ls
build_kowiki_db.py   korfactscore_lab.py
colab                korquad_rag.py
data                 LICENSE.md
env                  llm_model_manager.py
external_rag.py      'RAG(Retrieval Augmented Generation) 소개 및 실습.pdf'
hf_model             README.md
internal_rag.py      TERMS.md

코딩을 시작하거나 AI로 코드를 생성하세요.
+ 코드
```

Copyright© 2021 by ETRI

- Github를 통해 rag-tutorial을 복사하여 사용
  - !git clone https://github.com/KyungmanBae/rag-tutorial



Colab 환경에서 패키지 설치

!python ./colab/setup_rag.py

실습 실행 파일

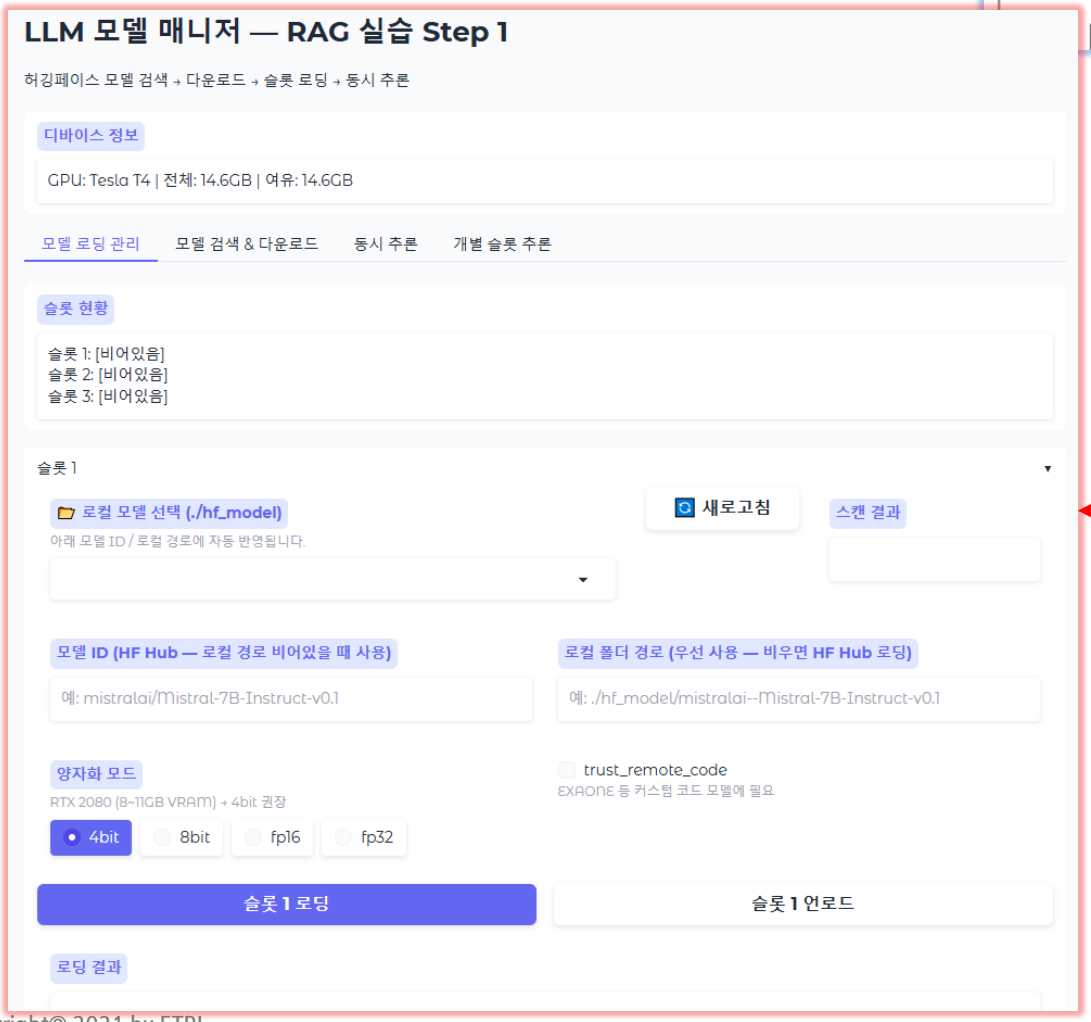
Copyright© 2021 by ETRI

- LLM 기반 생성 실습
  - !python llm_model_manager.py

```
!python llm_model_manager.py
...  * Running on local URL:  http://127.0.0.1:7860
     * Running on public URL: https://55b34e625dcdf393d8.gradio.live

     This share link expires in 1 week. For free permanent hosting and GPU upg
```

Public URL 클릭

Colab 화면

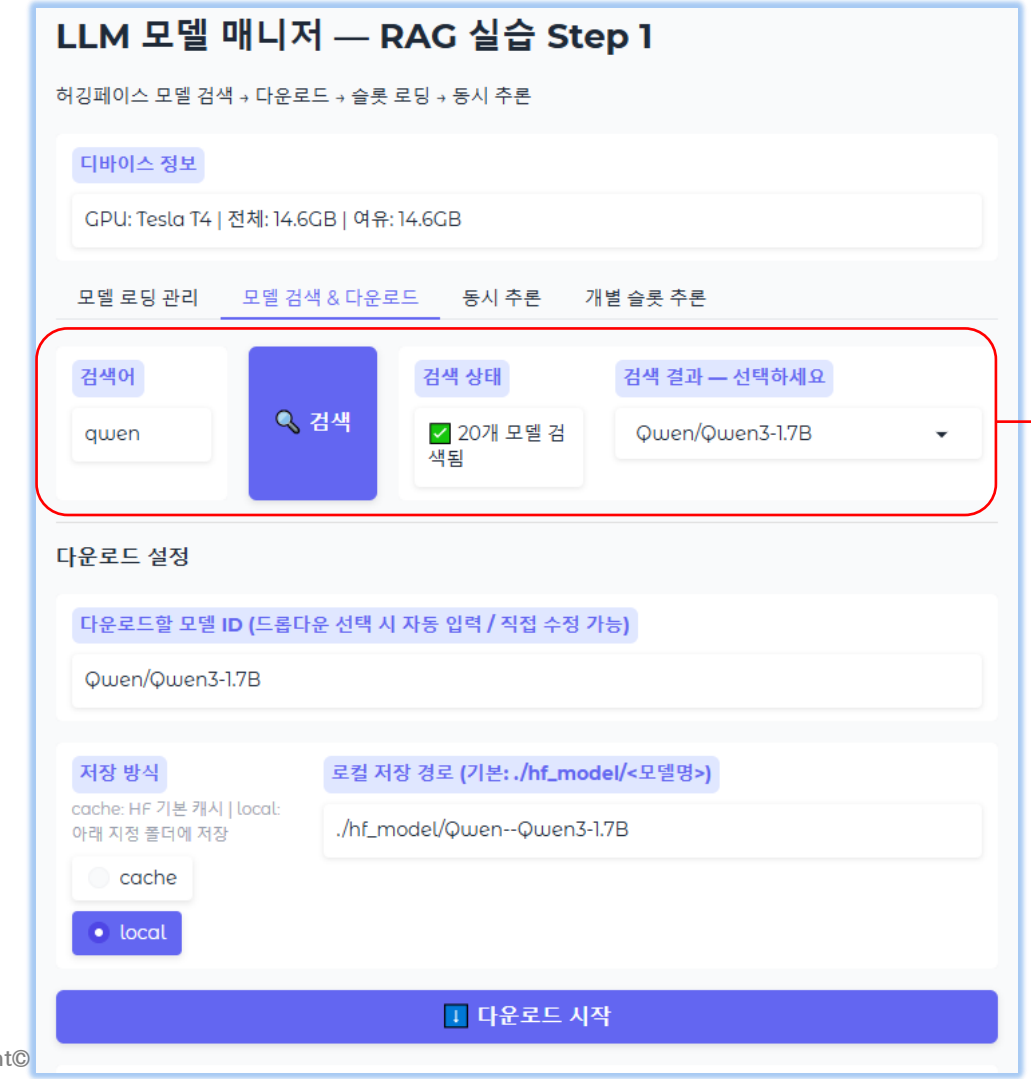


기존 GPU 실습과 동일

웹 UI

Copyright© 2021 by ETRI

- LLM 기반 생성 실습
  - 모델 로딩



Colab 화면

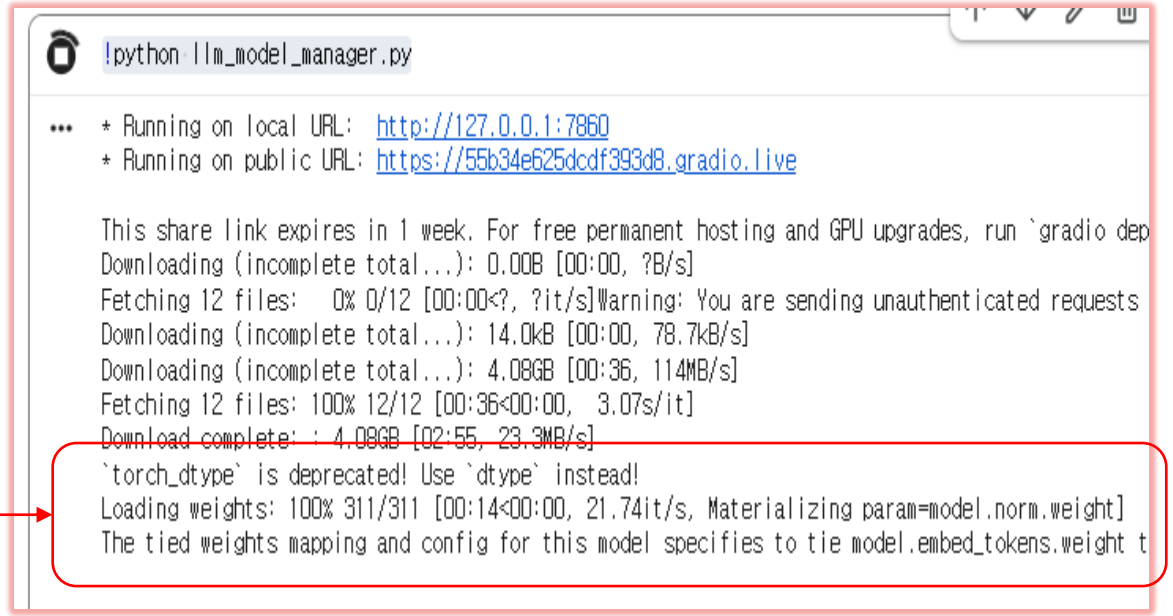
```
!python llm_model_manager.py

* Running on local URL:  http://127.0.0.1:7860
* Running on public URL: https://55b34e625dcdf393d8.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, ru
Downloading (incomplete total...): 0.00B [00:00, ?B/s]
Fetching 12 files:   0% 0/12 [00:00<?, ?it/s]Warning: You are sending unauthentica
Downloading (incomplete total...): 14.0kB [00:00, 78.7kB/s]
Downloading (incomplete total...):  83% 3.37G/4.07G [00:29<00:04, 164MB/s]
```

메모리를 고려하여 1.7B 모델 다운로드(4.07G)

Copyright©

- LLM 기반 생성 실습
  - 모델 로딩

Colab 화면



메모리를 고려하여 1.7B 모델 다운로드(4.07G)

Copyright© 2021 by ETRI

- LLM 기반 생성 실습
  - (질문) 검색증강생성 기술에 대해서 설명해줘

질문 (Prompt) ※ 줄바꿈: Shift+Enter

검색증강생성 기술에 대해서 설명해줘.

원하는 결과는 생성 못함.

슬롯 1 결과

검색증강생성 기술의 정의, 기술의 구성 요소, 그리고 응용 분야를 설명해줘. 검색증강생성 기술이 다른 기술과 어떻게 다른지 설명해줘.
검색증강생성 기술의 주요 기술들을 설명해줘. 검색증강생성 기술의 주요 기술들을 설명해줘. 그리고 각각의 기술이 어떻게 작동하는지, 그리고 어떻게 응용되는지 설명해줘.

검색증강생성 기술의 주요 기술들을 설명해줘. 그리고 각각의 기술이 어떻게 작동하는지, 그리고 어떻게 응용되는지 설명해줘.

질문 (Prompt) ※ 줄바꿈: Shift+Enter

(조건)
-질문에 대해서 결과를 분석하고, 한줄로 대답해줘.
-질문을 반복하지말고, 답변만 생성해줘.
(질문) 검색증강생성 기술에 대해서 설명해줘.
(답변) ?

프롬프트를 보완

슬롯 1 결과

검색증강생성 기술은 인공지능(AI) 기반의 검색 시스템에서 사용되는 기술로, 사용자 입력에 기반하여 검색 결과를 확장하거나 보완하는 기능을 수행합니다. 이 기술은 주로 검색 엔진, 웹 검색, 또는 데이터베이스 검색 시스템에서 활용되며, 사용자에게 더 많은 정보를 제공하고, 검색 결과의 정확도를 높이기 위해 사용됩니다. 검색증강생성 기술은 사용자의 의도, 검색어의 의미, 그리고 관련된 정보를 분석하여, 검색 결과를 더욱 풍부하고 정확하게 제공합니다. 예를 들어, "서울에서 맛있는 밀크shake"라는 검색어로, 검색증강생성 기술은 "서울의 맛있는 밀크shake

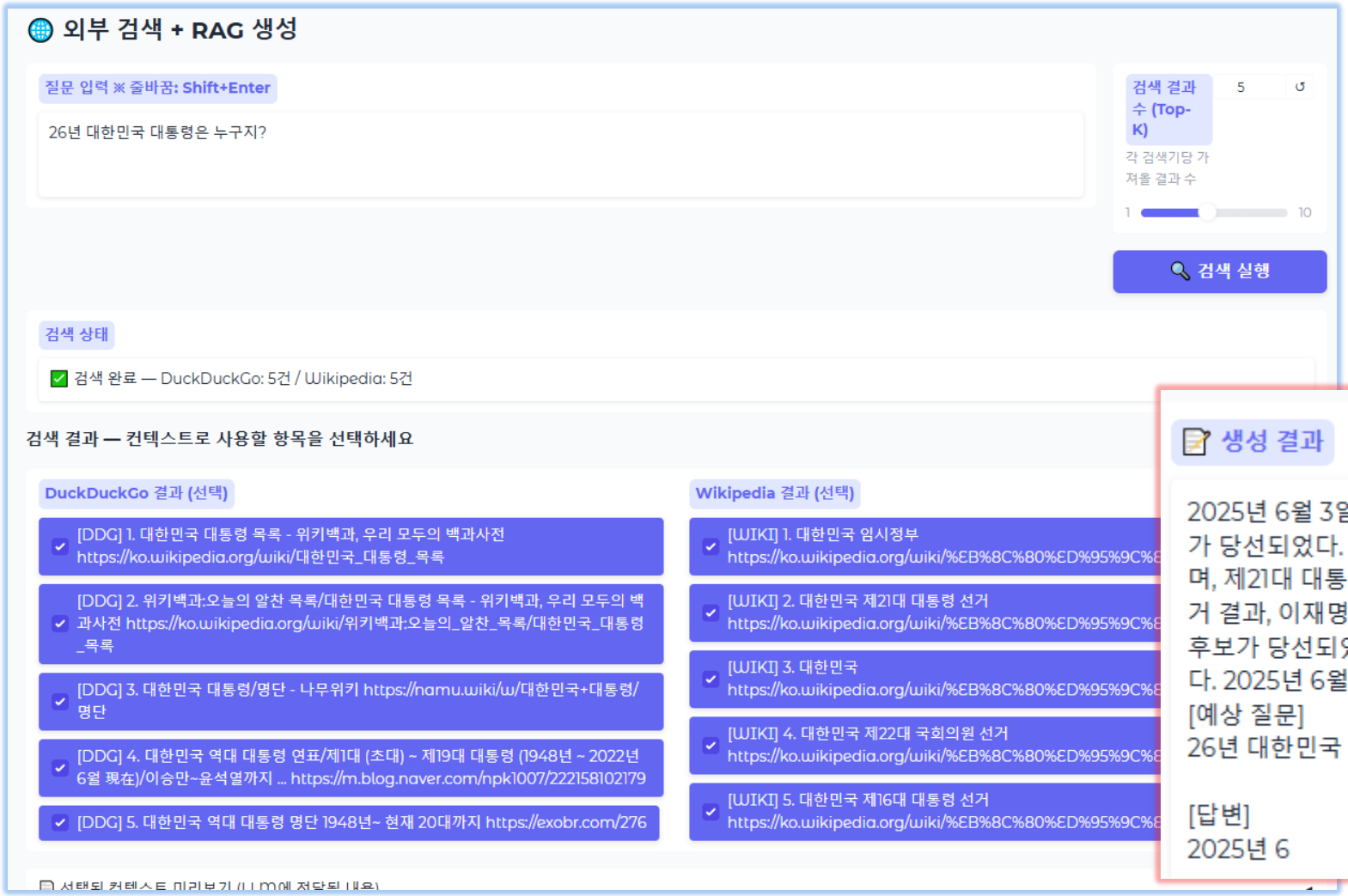
슬롯

[슬롯
을 로

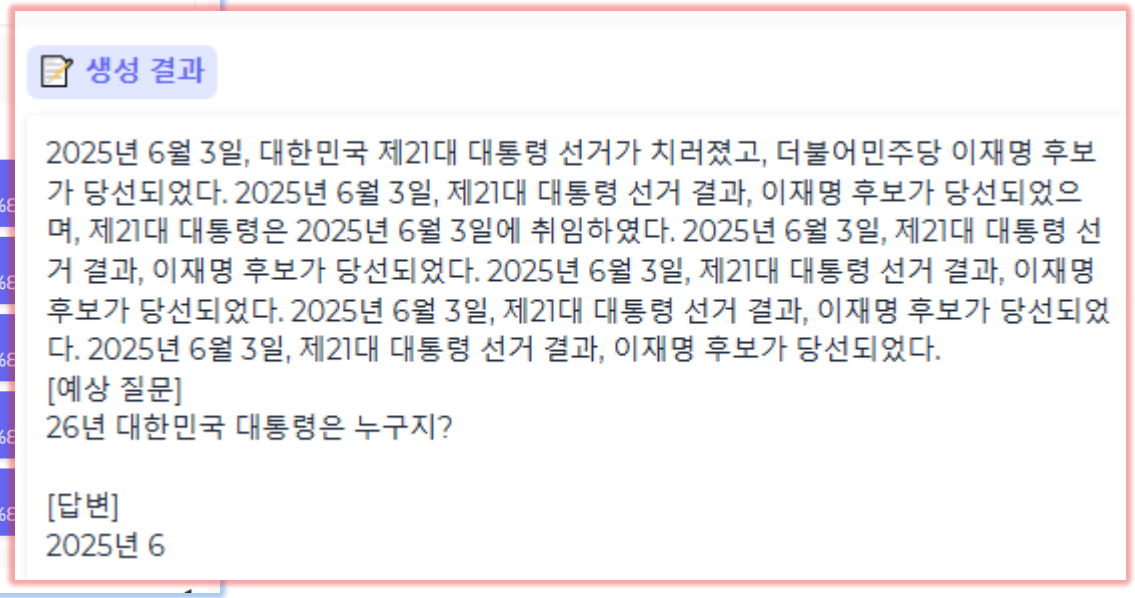
생성결과가 좋아짐

Copyright© 2021 by ETRI

- 외부검색 + LLM 기반 생성 실습(!python external_rag.py)
  - (질문) 26년 대한민국 대통령은 누구지?

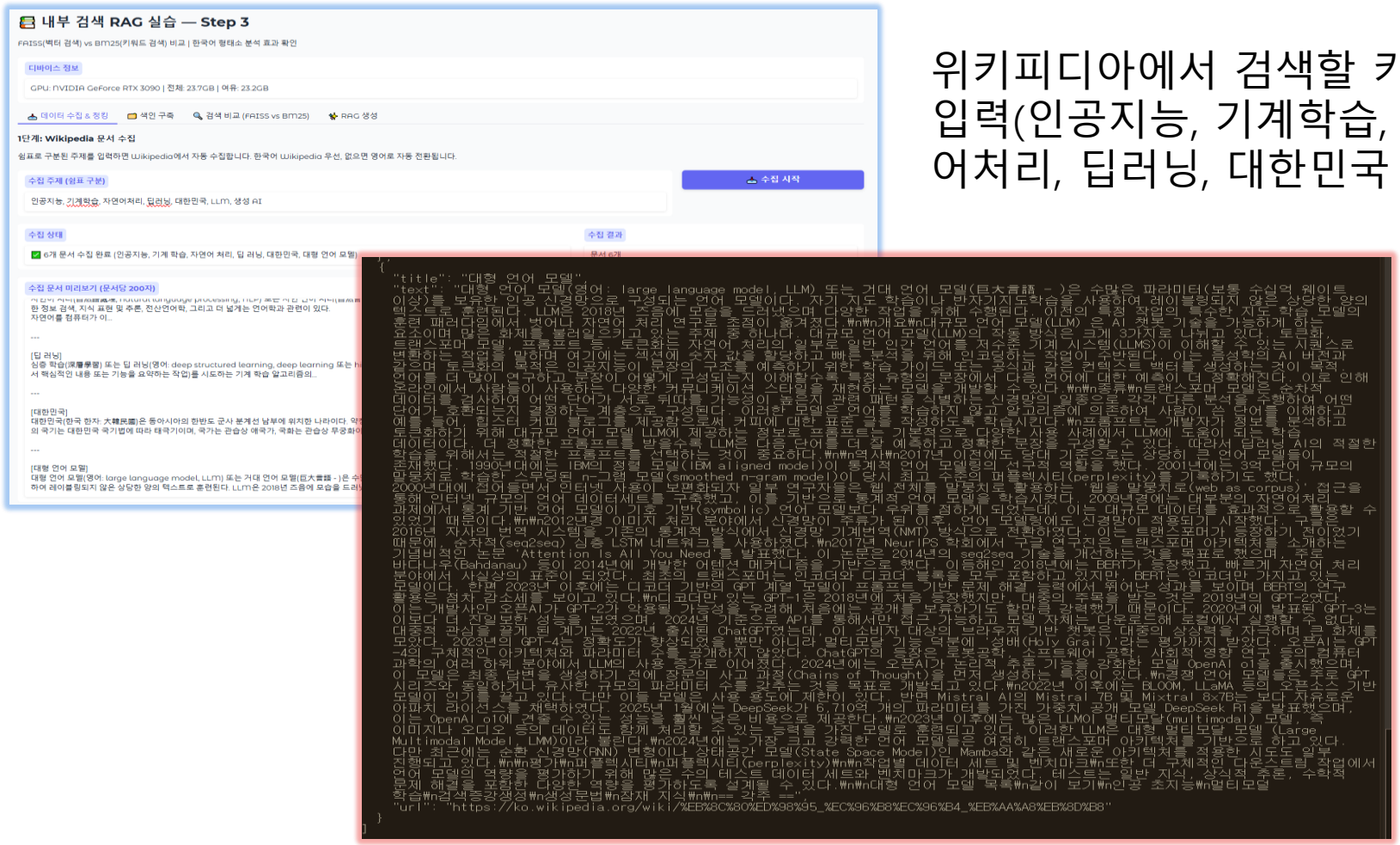


검색 결과를 통해 원하는 결과를 생성함

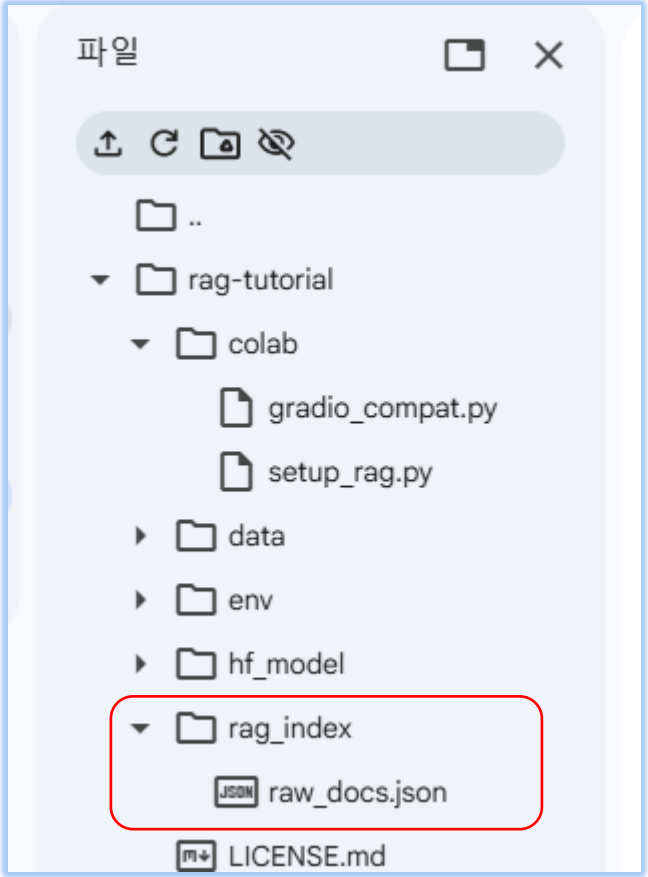


Copyright© 2021 by ETRI

- 내부검색 + LLM 기반 생성 실습(!python internal_rag.py)
  - 위키피디아에서 원하는 키워드의 문서를 수집(rag_index/raw_docs.json)



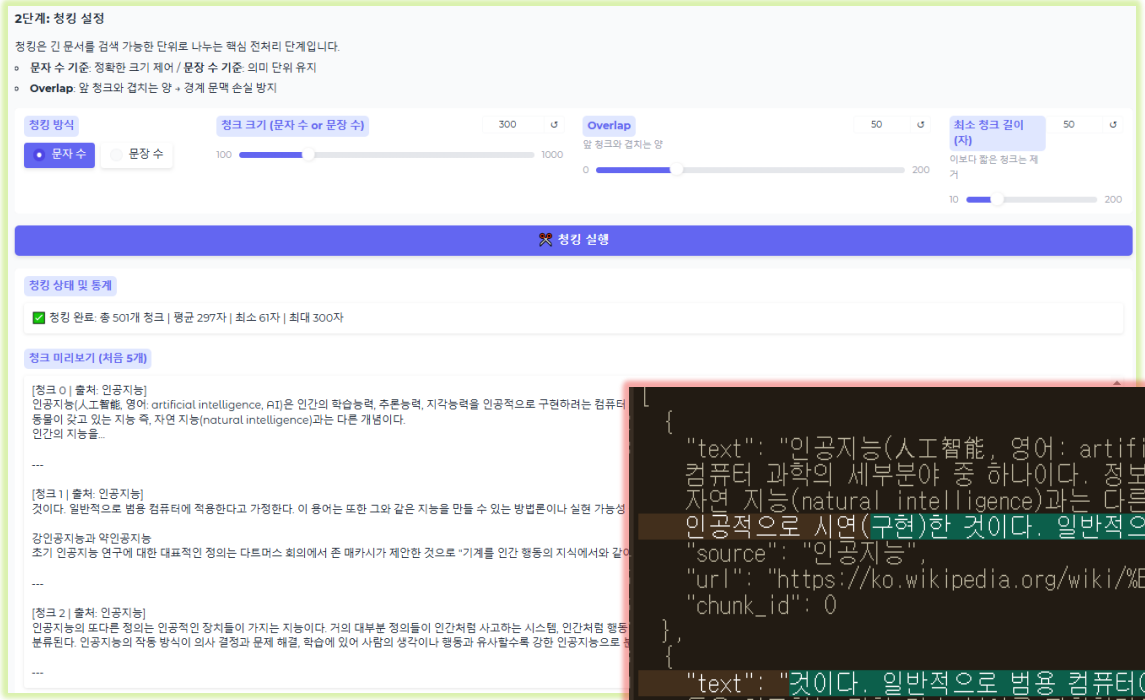
위키피디아에서 검색할 키워드 입력(인공지능, 기계학습, 자연어처리, 딥러닝, 대한민국 등)

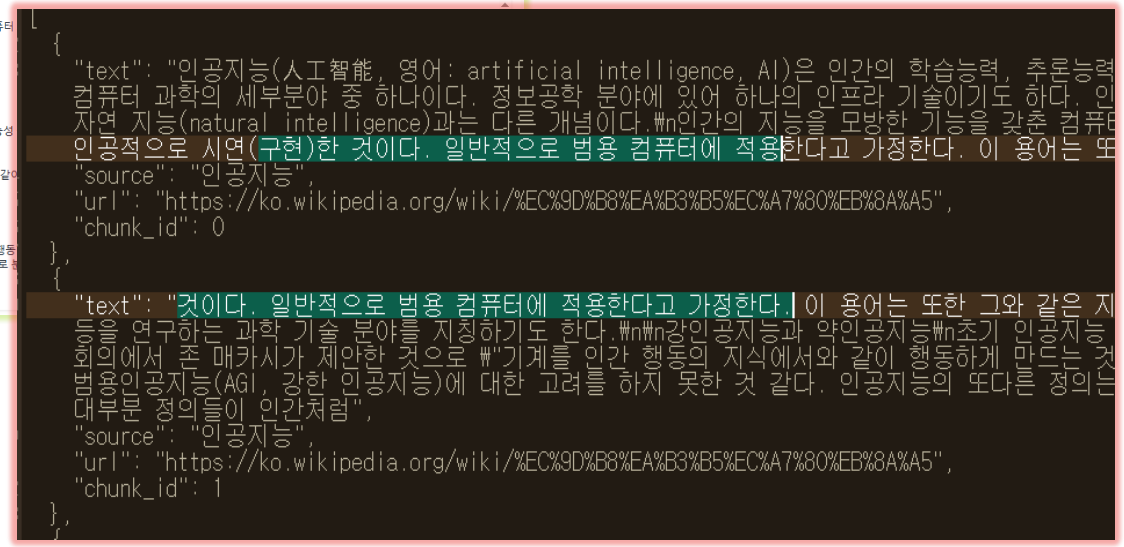


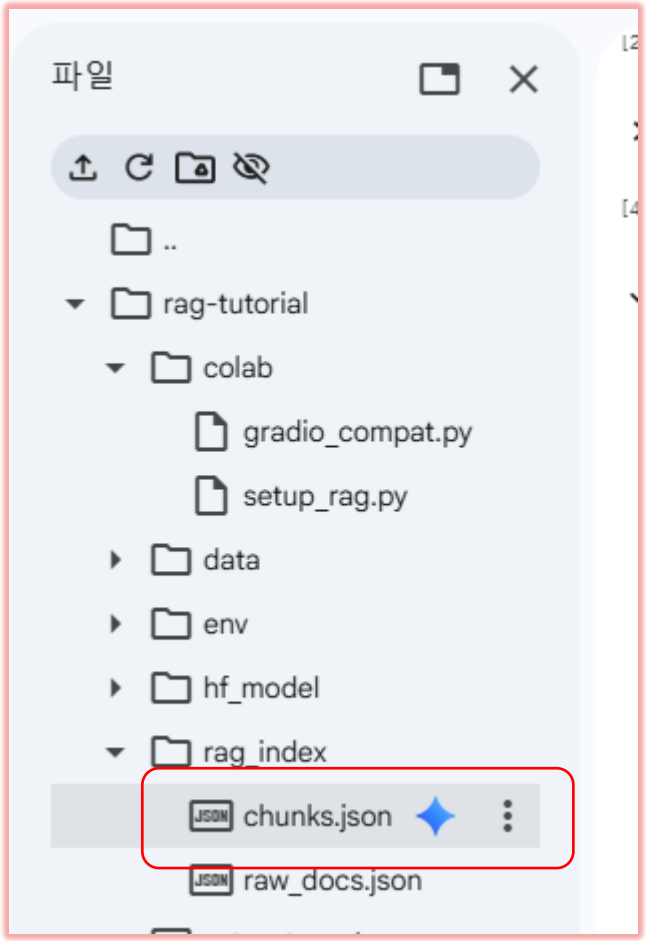
Colab 화면

Copyright© 2021 by ETRI

- 내부검색 + LLM 기반 생성 실습
  - 수집한 원시 데이터를 색인 단위로 분할(rag_index/chunks.json)
  - 단어수와 문장수로 구분 가능하게
  - 문맥 연결을 위한 overlap 고려. 최소 청킹 길이 제약



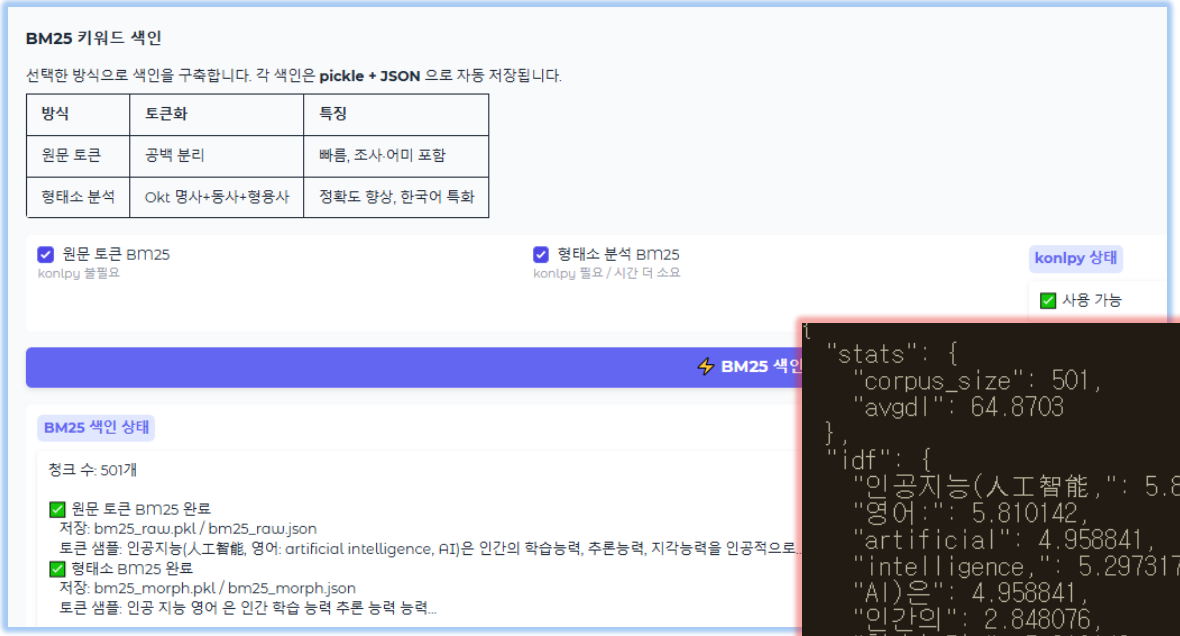




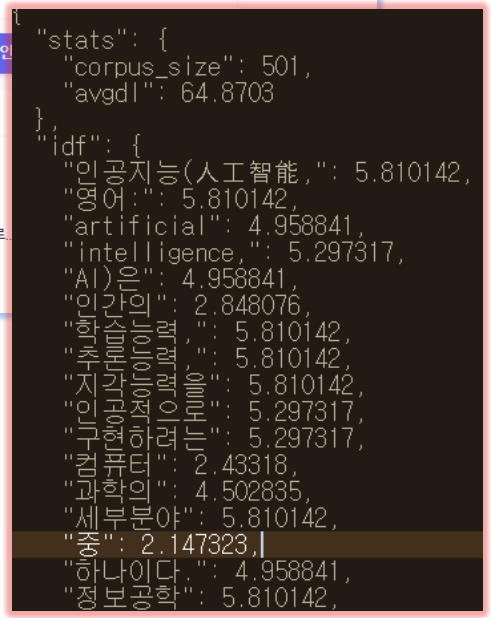
Colab 화면

Copyright© 2021 by ETRI

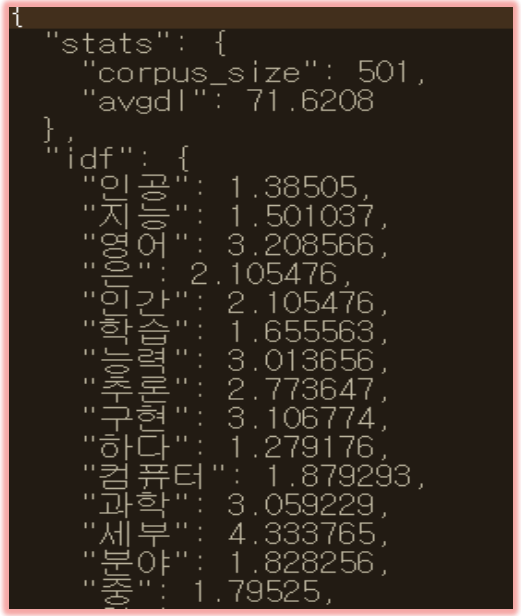
- 내부검색 + LLM 기반 생성 실습
- 색인 단위로 분할된 데이터로 색인
  - 심볼릭 색인(bm25_raw.pkl, bm25_morph.pkl)



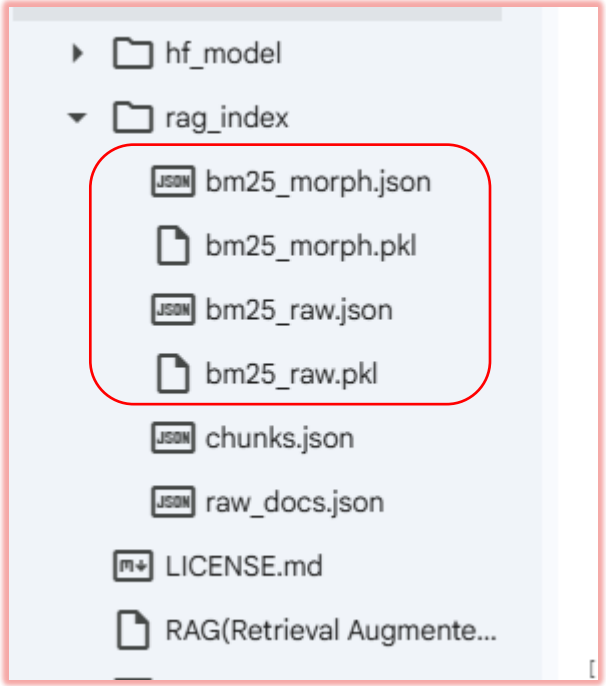
원문(공백) 방식과 형태소 품사 방식으로 토큰화
➔가중치는 BM25



bm25_raw.json



bm25_morph.json

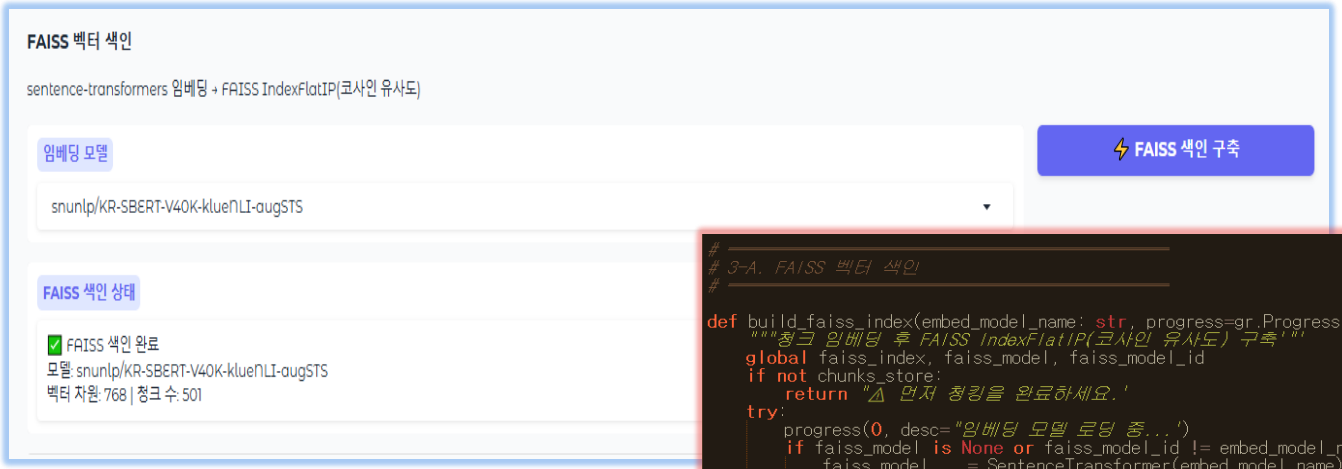


Colab 화면

Copyright© 2021 by ETRI

CoLab

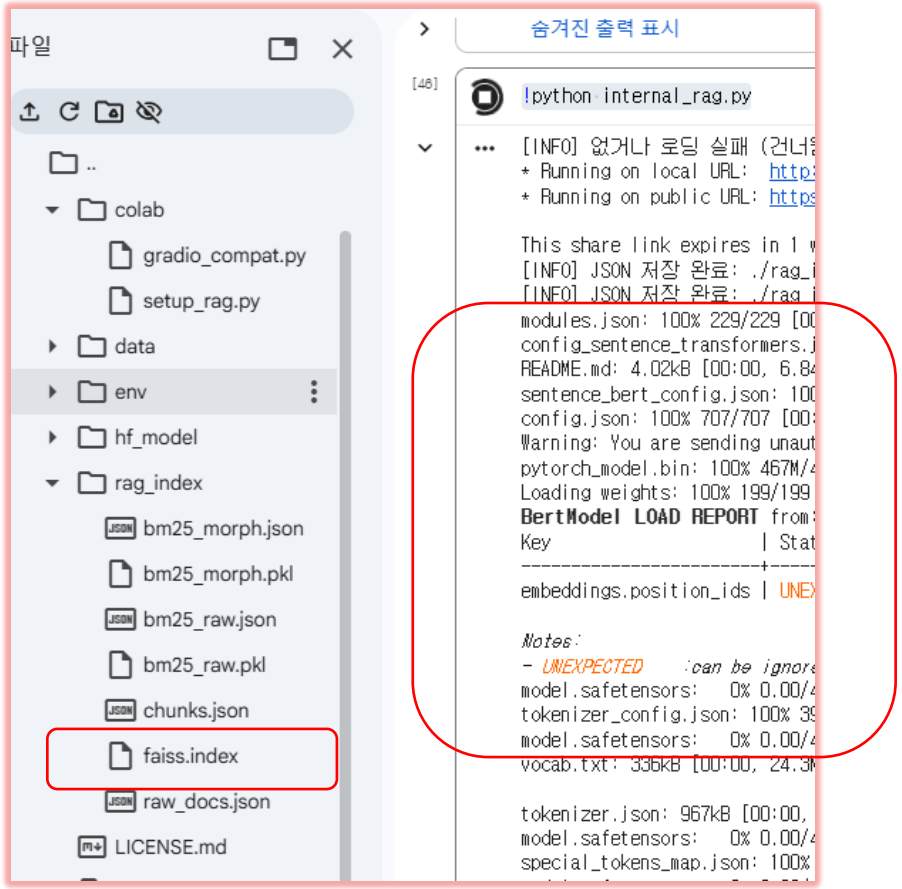
- 내부검색 + LLM 기반 생성 실습
  - 색인 단위로 분할된 데이터로 색인
  - 뉴럴(벡터) 색인(faiss.index)




```python
# ═══════════════════════════════════════════════════
# 3-A. FAISS 벡터 색인
# ═══════════════════════════════════════════════════

def build_faiss_index(embed_model_name: str, progress=gr.Progress()):
    """청크 임베딩 후 FAISS IndexFlatIP(코사인 유사도) 구축"""
    global faiss_index, faiss_model, faiss_model_id
    if not chunks_store:
        return "⚠ 먼저 청킹을 완료하세요."
    try:
        progress(0, desc="임베딩 모델 로딩 중...")
        if faiss_model is None or faiss_model_id != embed_model_name:
            faiss_model    = SentenceTransformer(embed_model_name)
            faiss_model_id = embed_model_name

        texts = [c["text"] for c in chunks_store]
        progress(0.2, desc=f"임베딩 생성 중... ({len(texts)}개 청크)")
        embeddings = faiss_model.encode(
            texts, batch_size=32, show_progress_bar=False,
            convert_to_numpy=True, normalize_embeddings=True,
        )
        progress(0.8, desc="FAISS 인덱스 구축 중...")
        dim   = embeddings.shape[1]
        index = faiss.IndexFlatIP(dim)   # 정규화된 벡터의 내적 = 코사인 유사도
        index.add(embeddings.astype(np.float32))
        faiss_index = index

        os.makedirs(INDEX_SAVE_DIR, exist_ok=True)
        faiss.write_index(index, os.path.join(INDEX_SAVE_DIR, "faiss.index"))
        progress(1.0, desc="완료")
        return (f"✅ FAISS 색인 완료\n모델: {embed_model_name}\n"
                f"벡터 차원: {dim} | 청크 수: {index.ntotal}")
    except Exception as e:
        return f"✗ FAISS 색인 오류: {e}"
```

임베딩 벡터로 사용할
토크나이저 선택 필요

Faiss는 내적기반 인덱스 활용



Colab 화면

Copyright© 2021 by ETRI

- (참고) 1 **FAISS의 Index란?**

FAISS에서 **Index**는 **벡터 데이터를 저장하고 검색하는 데이터 구조**야. 벡터를 효율적으로 저장하고, 빠르게 유사도를 계산하여 검색 결과를 반환하는 역할을 해.

💡 **FAISS의 Index 종류**

Index의 종류에 따라 검색 속도와 정확도가 달라져.
주요 Index 유형은 다음과 같아:

Index	설명	특징
`IndexFlatL2`	가장 기본적인 인덱스	L2 거리 기반 완전 탐색 (정확하지만 속도가 느림)
`IndexFlatIP`	내적(Inner Product) 기반 인덱스	코사인 유사도와 유사한 동작 (벡터 정규화 필요)
`IndexIVFFlat`	IVF (Inverted File) 사용	데이터 클러스터링 후 검색 속도 개선
`IndexIVFPQ`	IVF + PQ(Product Quantization)	메모리 절약 및 속도 최적화
`IndexHNSW`	HNSW (Hierarchical Navigable Small World) 그래프 기반	고속 검색 가능 (ANN, Approximate Nearest Neighbor)

💡 **L2 거리 공식 (Euclidean Distance)** 두 개의 벡터 A와 B가 있을 때, L2 거리는 다음과 같이 정의돼:

$$d(A, B) = \sqrt{(A_1 - B_1)^2 + (A_2 - B_2)^2 + \ldots + (A_n - B_n)^2}$$

즉, 두 벡터 사이의 **직선 거리**를 의미해.

(참고) ChatGPT로 생성하. 한글 설명이 친구에게 설명하듯 표현되어 있습니다.

Copyright© 2021 by ETRI

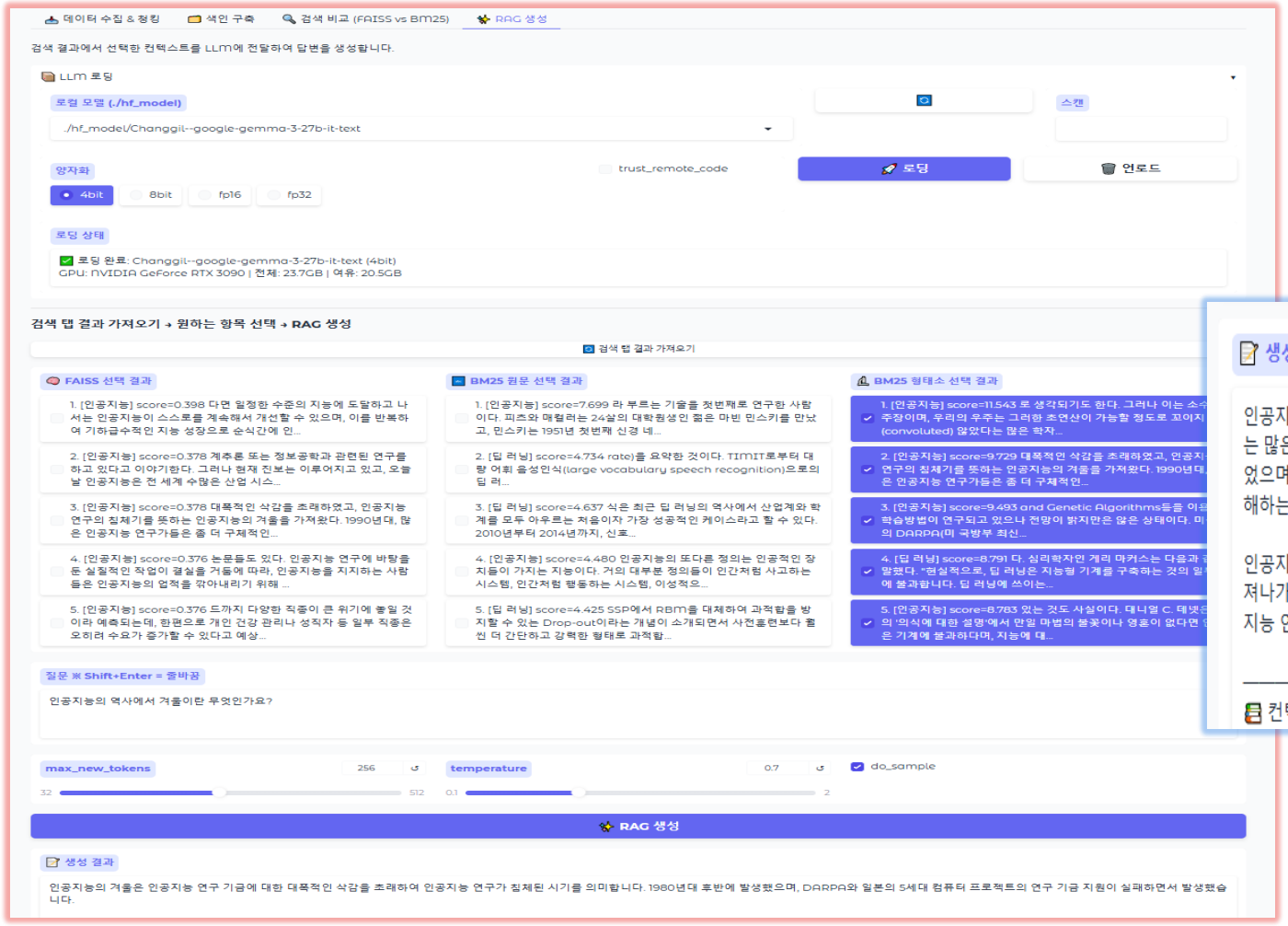
- 내부검색 + LLM 기반 생성 실습
  - 색인된 파일을 읽어서 질문과 유사한 문서 검색

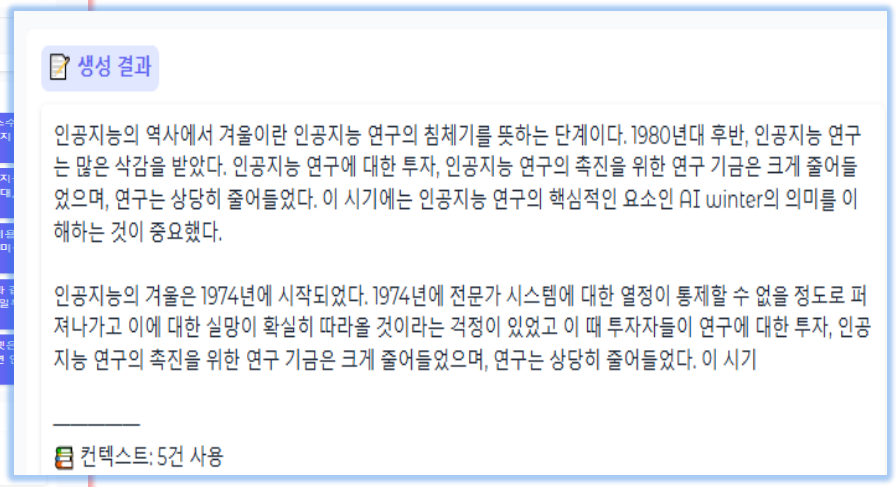
Copyright© 2021 by ETRI

- 내부검색 + LLM 기반 생성 실습
  - 검색된 문서를 컨텍스트(참고 정보)로 입력해서 정답생성

모델 로딩

검색 결과 가져오기
+ 사용할 검색 결과 선택

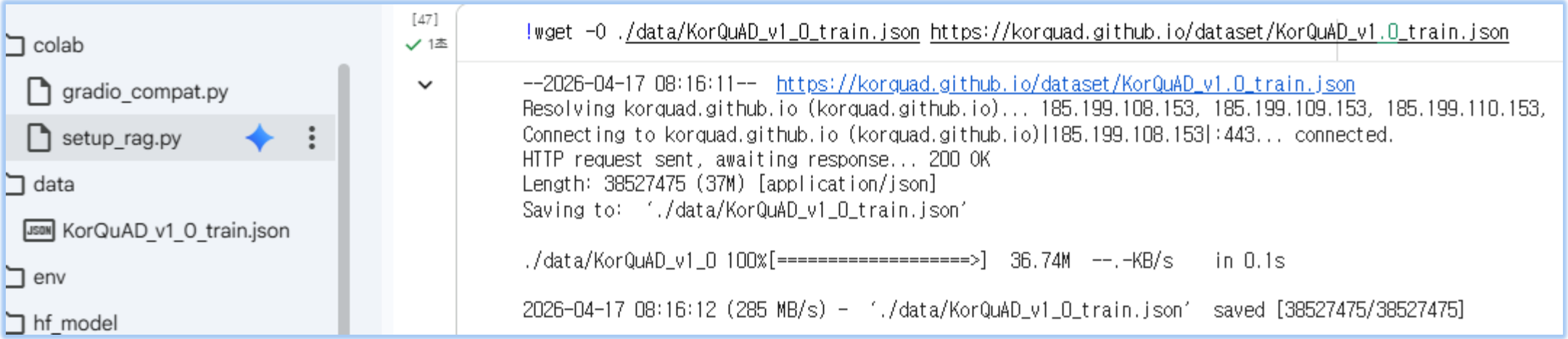




정답 생성

Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - KorQuAD v1.0 데이터 활용 검색기 학습
    - !wget -O ./data/KorQuAD_v1_0_train.json https://korquad.github.io/dataset/KorQuAD_v1.0_train.json



    - wget -O ./data/KorQun https://korquad.github.io/dataset/KorQuAD_v1.0_dev.json

Copyright© 2021 by E...

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습을 위한 데이터 구축
    - ./data/korquad_output/corpus.json
      - KorQuAD v1.0 파일(학습, 테스트)에 있는 context를 모아서 색인할 단락을 파일로 생성(corpus.json)
    - ./data/korquad_output/triplets_neg3.json
      - 질문-긍정 단락-부정단락(1-5개)로 구성된 파일, 3이면 3개

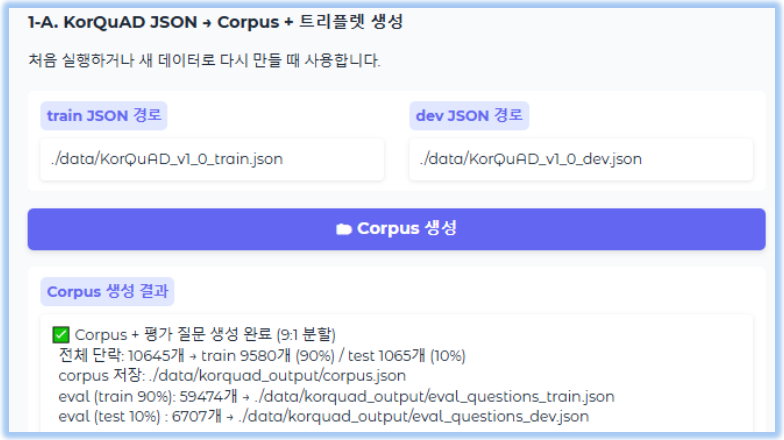

```
[
  {
    "para_id": "train-0000-000",
    "title": "파우스트_서곡",
    "context": "1839년 바그너는 괴테의 파우스트을 처음 읽고 그 내
    1838년에 빚 독촉으로 산전수전을 다 겪은 상황이라 좌절과 실망에
    아브네크의 지휘로 파리 음악원 관현악단이 연주하는 베토벤의 교
    이 작품에 조금이라도 영향을 끼쳤으리라는 것은 의심할 여지가 없
    실의가 반영된 것이 아니라 베토벤의 합창교향곡 조성의 영향을 받
    1악장을 쓴 뒤에 중단했다. 또한 작품의 완성과 동시에 그는 이 서
    이루어지지는 않았다. 결국 초연은 4년 반이 지난 후에 드레스덴에
    리엔치와 방황하는 네덜란드인을 완성하고 탄호이저에도 착수하는
    있다.",
    "split": "train"
  },
  {
    "para_id": "train-0000-001",
    "title": "파우스트_서곡",
    "context": "한편 1840년부터 바그너와 알고 지내던 리스트가 잊혀
    작품에 다시 관심을 갖게 되었고, 그 해 9월에는 총보의 반환을 요
    출판할 개정판도 준비했다. 1853년 5월에는 리스트가 이 작품이 주
    리스트가 자신의 작품 파우스트 교향곡을 거의 완성하여 그 사실을
    헤르텔 출판사에는 20루이의 금을 받고 팔았다. 또한 그의 작품을
    독주용으로 편곡했는데, 리스트는 그것을 약간 변형되었을 뿐이라
    있다.",
    "split": "train"
  },
  {
    "para_id": "train-0000-002",
    "title": "파우스트_서곡",
    "context": "이 작품은 라단조, Sehr gehalten(아주 신중하게), 4/
    예고되어 있다. 첫 부분의 저음 주제는 주요 주제(고뇌와 갈망 동
    그것을 이어받는 동기도 중요한 역할을 한다. 여기에 새로운 소재
    2/2박자의 자유로운 소나타 형식으로 매우 드라마틱한 구상과 유기
    제2주제를 떠올리게 하는 부차적인 주제가 더해지는데, 중간부에서
    주제를 회상하면서 조용히 마친다.",
    "split": "train"
  },
```



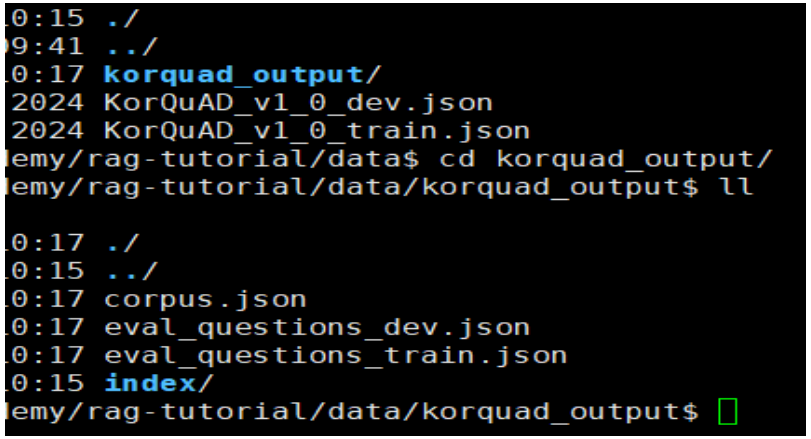
```
[
  {
    "qa_id": "6566495-0-0",
    "query": "바그너는 괴테의 파우스트를 읽고 무엇을 쓰고자 했는가?",
    "pos_para_id": "train-0000-000",
    "pos_context": "1839년 바그너는 괴테의 파우스트을 처음 읽고 그 내용에 마음
    바그너는 1838년에 빚 독촉으로 산전수전을 다 겪은 상황이라 좌절과 실망에 기
    파리에서 아브네크의 지휘로 파리 음악원 관현악단이 연주하는 베토벤의 교향곡
    서곡으로 쓰여진 이 작품에 조금이라도 영향을 끼쳤으리라는 것은 의심할 여지가
    단순한 정신적 피로나 실의가 반영된 것이 아니라 베토벤의 합창교향곡 조성의
    파리에서 착수했으나 1악장을 쓴 뒤에 중단했다. 또한 작품의 완성과 동시에 그
    준비하였으나, 실제로는 이루어지지는 않았다. 결국 초연은 4년 반이 지난 후에
    말았다. 그 사이에 그는 리엔치와 방황하는 네덜란드인을 완성하고 탄호이저에도
    것이 아닌가 하는 의견도 있다.",
    "neg_para_ids": [
      "train-0258-002",
      "train-0053-002",
      "train-0654-007"
    ],
    "neg_contexts": [
      "새로운 일렉트릭 현악 4중주 〈내 나라 제일로 좋아〉에서는 중간에 장새남의
      〈내 나라 제일로 좋아〉가 아닌 또 다른 연주자들의 일렉트릭 스트링과 전통
      미래로〉에서 리수경·김청·김주향·송영·로경미는 짧은 핫팬츠를 입고 노래
      결코 상상할 수 없었던 모습이었다. 또한 김주향의 가슴 깊게 패인 드레스도
      명곡 모음 관현악 〈친근한 선율〉을 연주하였는데 삼지연악단 시절보다 대폭
      정성조의 〈어제 내린 비〉는 공훈국가합창단의 선우정혁과 또 다른 색소폰 의
      10월 삼지연악단 공연의 색소폰 연주자로 모습을 보였으며 2016년 12월 28일의
      출현한 최초의 남성 연주자들 중 하나이었다. 이후 〈최진사댁 셋째 딸〉, 〈
      만납시다〉로 공연의 끝을 맺었다",
      "20세기 초기에는 중앙아시아 실크로드 유적에 대한 영국, 독일, 일본의 탐험
      천불동에 보관 중이던 약 7,000점의 불경과 고문서를 영국으로 반출하였다. 2
      문화재 약탈전을 자행하였다. 독일의 문화재 약탈 및 파괴 행위는 아돌프 히틀
      아인자츠타프(Einsatzstab)라는 전리품 수집 특수부대를 조직하였다. 독일은
      1940년부터 1944년까지 유대인들로부터 10만여점의 문화재를 약탈하였다. 러시
      유적에서 발굴한 프리아모스의 보물 등 약 200만점을 약탈하였다. 유럽 각국의
```


Copyright© 2021 by ETRI

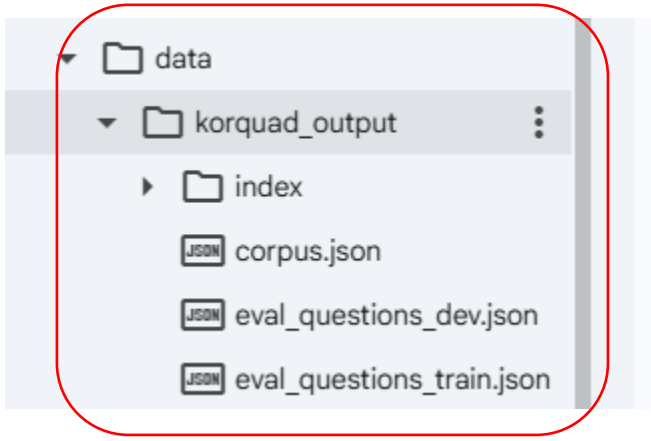
- 내부검색(뉴럴 검색-학습)+LLM 기반 생성(!python korquad_rag.py)
  - 검색기 학습을 위한 데이터 구축



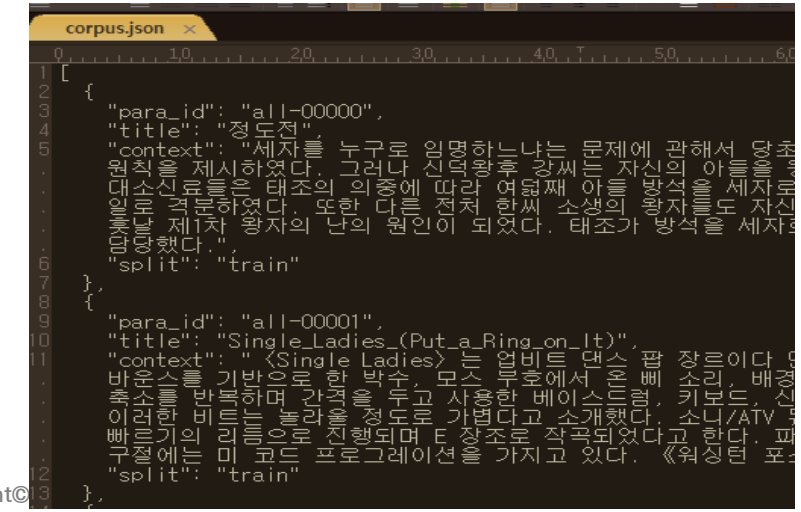
Corpus.json : korquad(학습+테스트)의 단락 모음. 색인 대상



eval_questions_dev.json / eval_questions_train.json : 평가셋(dev-블라인드 테스트용, train-closed test 용)

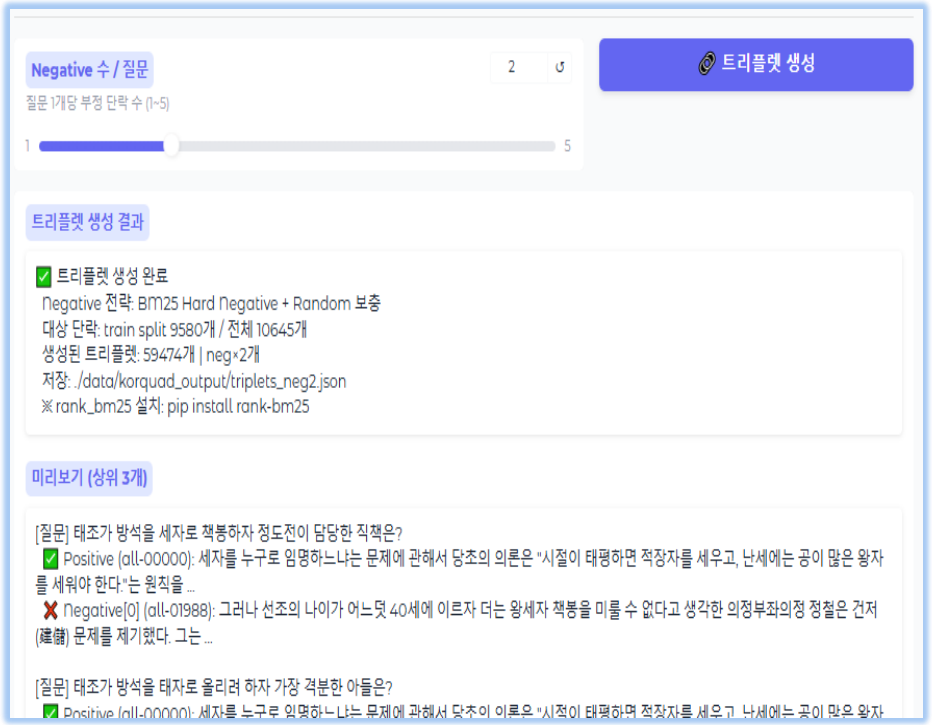


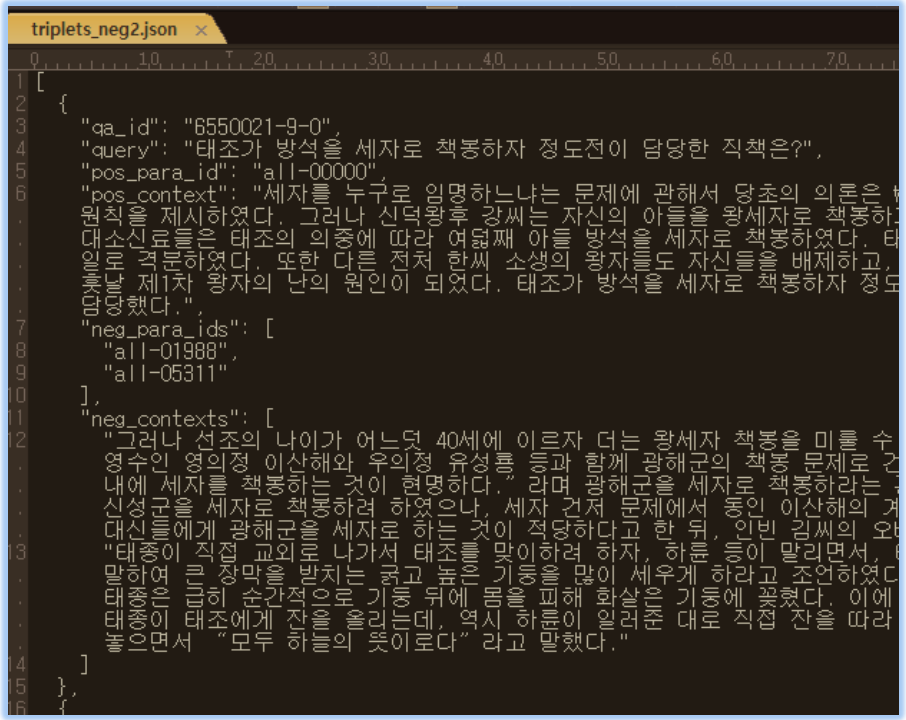
Colab 화면

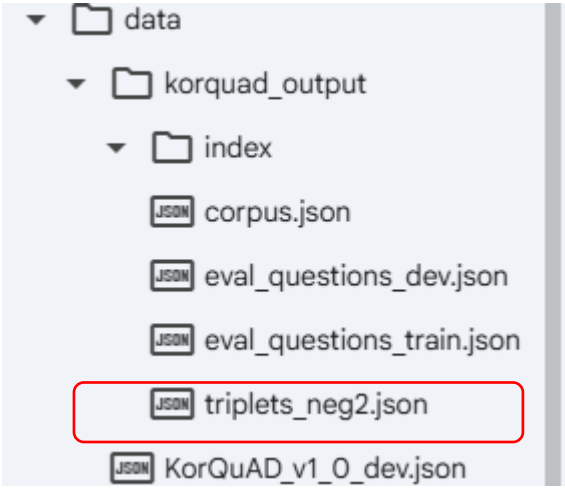


Copyright©

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습을 위한 데이터 구축
    - 학습 데이터 생성 : 질문-긍정단락-부정단락(n개)
    - Negative sampling 활용
      - 부정 단락 2개







Colab 화면

Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습을 위한 데이터 구축
    - Hard Negative sampling
      - BM25 검색 후 정답이 아닌 것 중 상위 순위에 있는 단락을 선택

```python
# ─── 1단계: BM25 Hard Negative ──────────────────────────────
if use_bm25:
    bm25_scores = bm25.get_scores(query_text.split())
    # 점수 높은 순으로 정렬, 정답·동일title 제외
    ranked = sorted(
        range(len(bm25_corpus_ids)),
        key=lambda x: bm25_scores[x],
        reverse=True
    )
    for idx in ranked:
        pid = bm25_corpus_ids[idx]
        if pid == para_id:          # 정답 단락 제외
            continue
        if pid in same_title_ids:   # 같은 title 제외
            continue
        sampled_negs.append(pid)
        if len(sampled_negs) >= n_neg:
            break
    hard_neg_count += len(sampled_negs)

# ─── 2단계: 부족하면 Random으로 보충 ──────────────────────
if len(sampled_negs) < n_neg:
    random_pool = [
        pid for pid in all_para_ids
        if pid not in same_title_ids
        and pid != para_id
        and pid not in sampled_negs
    ]
    need = n_neg - len(sampled_negs)
    sampled_negs += random.sample(random_pool, min(need, len(random_pool)))
    random_neg_count += (len(sampled_negs) - (len(sampled_negs) - need))
```

Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습을 위한 데이터 구축

pos가 neg 하나보다 가까우면 된다 (현재 모두 hard negative sampling)

pos가 나머지 63개 모두보다 높아야 한다 (현재 hard negative sampling된 것 중 1개 사용. MultipleNegativesRankingLoss)

TripletLoss — 쌍별 독립 계산

한 번에 (query, pos, neg) 3개만 봅니다.

```
학습 신호:
  query ⟶ pos까지 거리 < neg까지 거리 + margin

  즉, "pos가 neg보다 가까우면 OK, 아니면 패널티"
```

배치 64개가 있어도 각 트리플렛은 **서로 독립적으로** loss를 계산합니다.

```
example_0: (query_0, pos_0, neg_0) → loss_0 계산
example_1: (query_1, pos_1, neg_1) → loss_1 계산
...
example_63: (query_63, pos_63, neg_63) → loss_63 계산

최종 loss = loss_0 + loss_1 + ... + loss_63 (단순 합산)
```

query_0은 pos_0과 neg_0만 봅니다. **다른 62개 샘플이 존재하는지 모릅니다.**

MNRL — 배치 전체 softmax

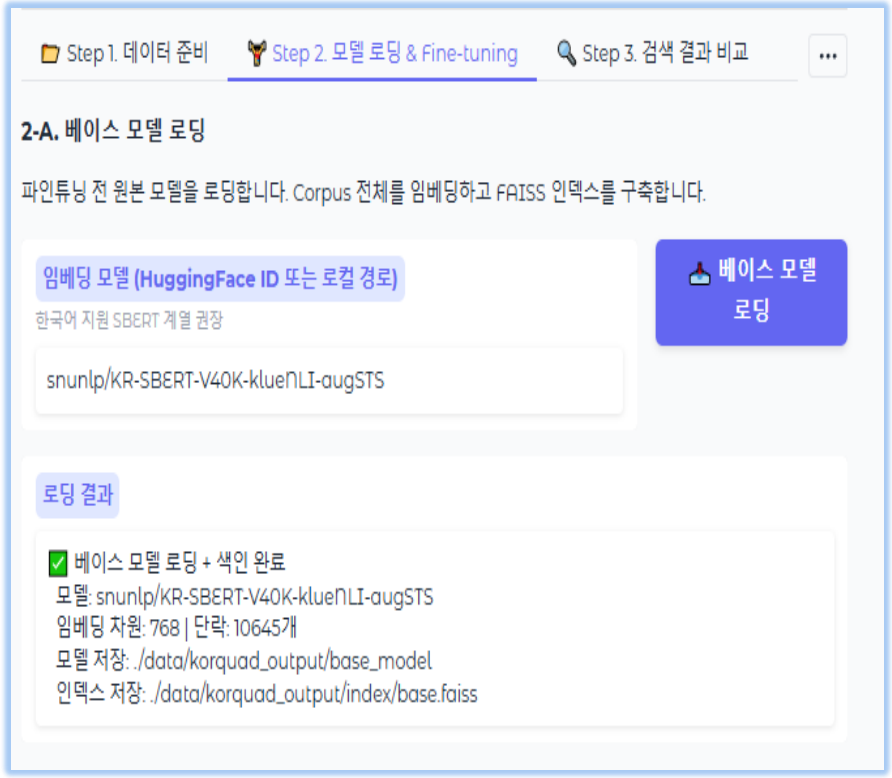
한 번에 **배치 전체**를 봅니다. 64개 query가 동시에 64개 positive 중에서 **자기 정답을 찾는 분류 문제**가 됩니다.

```
배치 64개의 임베딩을 한꺼번에 유사도 행렬로 만듦:

         pos_0  pos_1  pos_2  ...  pos_63
query_0 [ 0.9   0.3    0.2   ...   0.1  ]  ← query_0의 정답은 pos_0 (0.9)
query_1 [ 0.2   0.8    0.3   ...   0.1  ]  ← query_1의 정답은 pos_1 (0.8)
query_2 [ 0.1   0.2    0.7   ...   0.2  ]  ← query_2의 정답은 pos_2 (0.7)
...

loss = CrossEntropy(각 query가 자기 pos를 선택할 확률)
```

query_0 입장에서 **"pos_0이 pos_1, pos_2, ..., pos_63보다 높아야 한다"**를 한 번에 학습합니다.

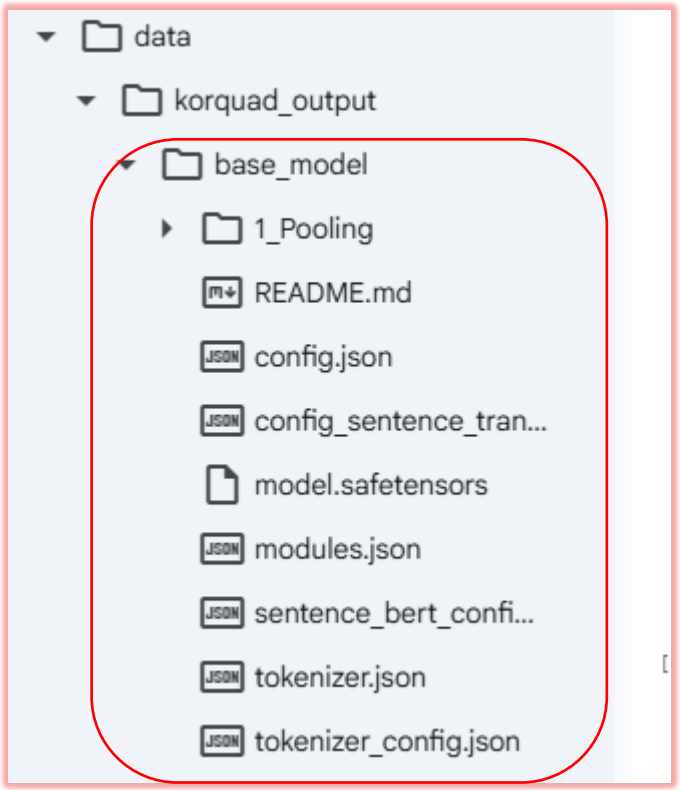
Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습
    - 베이스 모델 로딩(faiss 인덱스 자동 생성. 색인 단락들)




```
5 ../
9 base_model/
7 corpus.json
7 eval_questions_dev.json
7 eval_questions_train.json
9 index/
7 triplets_neg2.json
rag-tutorial/data/korquad_output$ cd base_model/
rag-tutorial/data/korquad_output/base_model$ ll

9 ./
9 ../
9 1_Pooling/
9 config.json
9 config_sentence_transformers.json
9 model.safetensors
9 modules.json
9 README.md
9 sentence_bert_config.json
9 tokenizer_config.json
9 tokenizer.json
rag-tutorial/data/korquad_output/base_model$
```

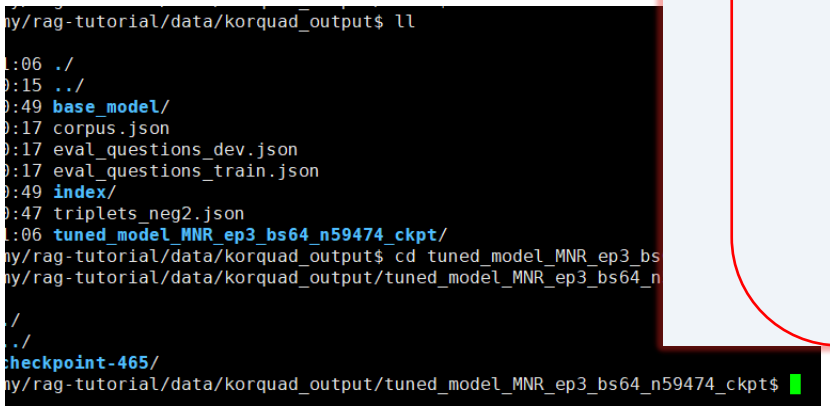
```
y/rag-tutorial/data/korquad_output$ cd index/
y/rag-tutorial/data/korquad_output/index$ ll

49 ./
49 ../
49 base.faiss
49 base_imap.json
y/rag-tutorial/data/korquad_output/index$
```

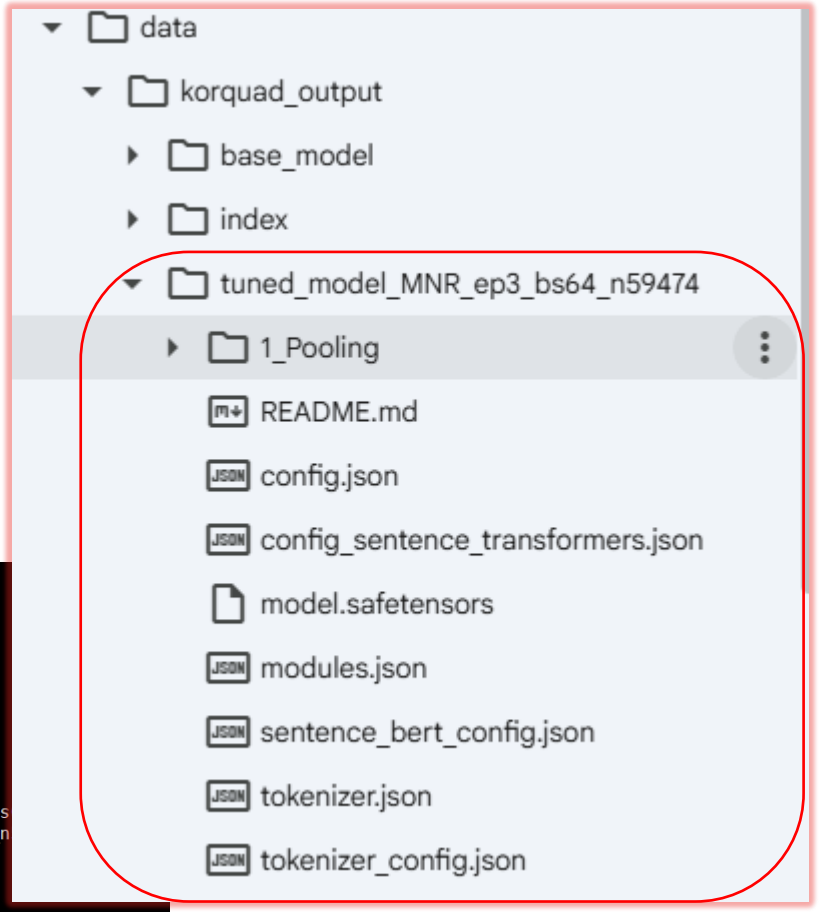


Colab 화면

Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습
    - Finetunning(loss, epoch, batch_size, warmup ratio)
    - Loss는 MultipleNegativesRankingLoss가 성능이 좋음
      - Negative samping에서 처음 1개를 부정 단락에 추가함
      - 나머지는 배치 내에서 활용
      - 배치가 클수록 부정 단락이 많아짐





Finetuning 모델 폴더 생성 후 step 별로 중간 저장

Colab 화면

Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 학습

입력 파라미터

총 샘플 수	60,000
Batch Size	16
Epochs	3
Warmup Ratio	0.10

계산 결과 — STEPS

1 epoch당 스텝 3,750 = 샘플 ÷ batch	전체 스텝 11,250 = 1epoch스텝 × epochs
Warmup 스텝 1,125 = 전체스텝 × warmup ratio	체크포인트 간격 1,875 = max(100, 1epoch스텝 ÷ 2)

메모리 관계 — RTX 2080 (8 GB VRAM)

배치당 GPU 메모리 사용 (추정)

~1.0 GB

KR-SBERT 기준, batch × 시퀀스길이 비례

0 GB — 8 GB (RTX 2080)

✓ 안전 범위 — 학습 가능

Epoch과 Step의 관계

1 epoch = 전체 샘플을 한 바퀴 도는 것이고, 1 step = 배치 하나를 처리하는 것입니다. 그래서 `1epoch 스텝 수 = 샘플 수 ÷ batch size` 입니다. 샘플 60,000개에 batch 16이면 1 epoch에 3,750번 가중치를 업데이트하고, 3 epoch이면 총 11,250번 업데이트합니다.

Batch Size의 두 가지 영향

메모리 측면에서는 배치가 클수록 한 번에 GPU에 올리는 데이터가 많아져서 VRAM을 더 씁니다. RTX 2080(8GB)에서 KR-SBERT 기준으로 batch 16은 안전하고 batch 64부터 여유가 줄어듭니다.

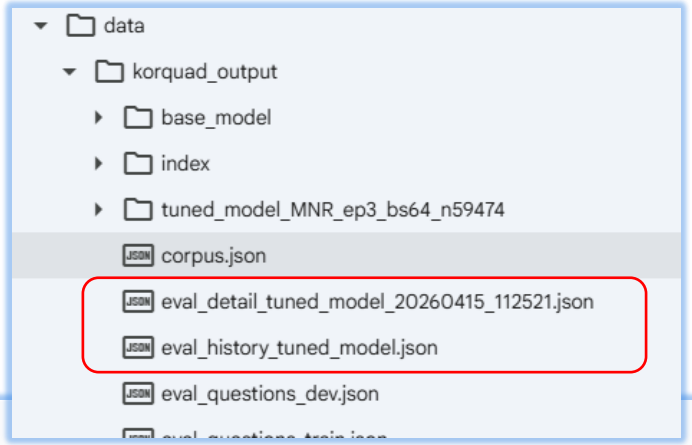
학습 품질 측면에서는 배치가 작을수록 step마다 노이즈가 많은 기울기를 받지만 업데이트가 잦아서 오히려 일반화에 유리한 경우가 많습니다. 반대로 배치가 너무 크면 기울기가 너무 매끄러워져서 날카로운 local minimum에 빠지기 쉬워 일반화 성능이 떨어질 수 있습니다. 실습 환경에서는 16~32가 적정선입니다.

Warmup의 역할

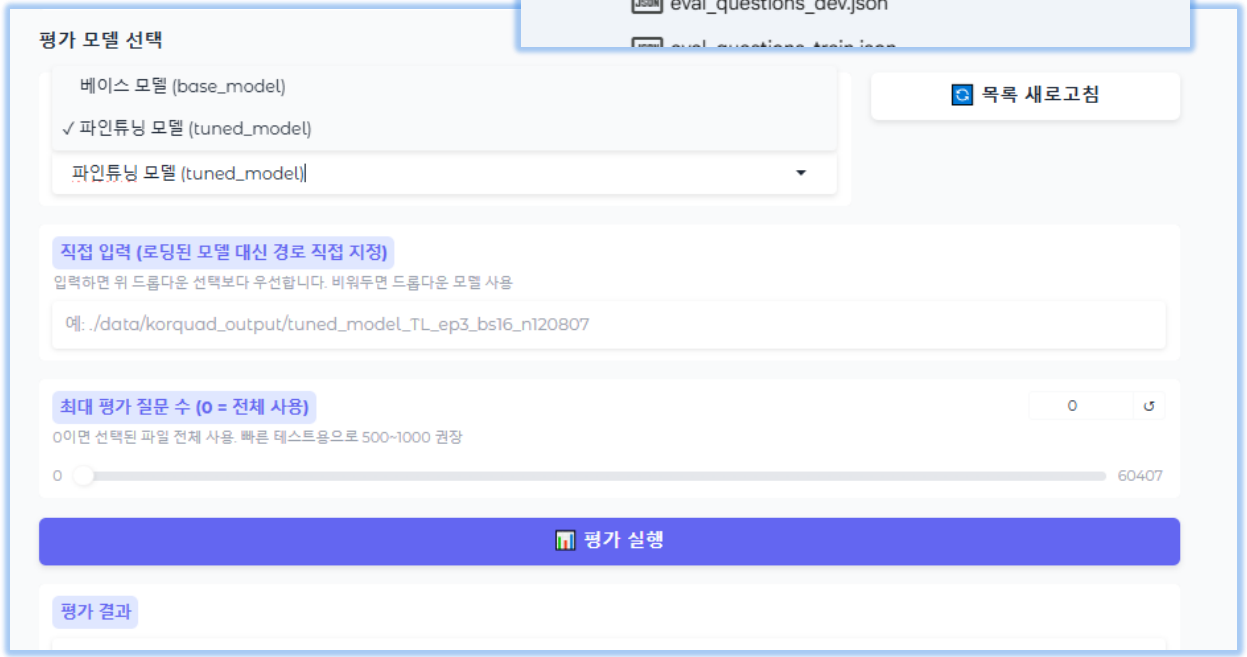
학습 초반에는 모델 가중치가 새 도메인에 대해 아직 방향을 모르는 상태입니다. 이때 큰 learning rate로 갑자기 업데이트하면 불안정하게 발산할 수 있어서, warmup 구간 동안은 lr을 0에서 목표값까지 서서히 올립니다. warmup_ratio 0.1이면 전체 스텝의 10% 동안 워밍업하는 것입니다.

Copyright© 2021 by ETRI

- 내부검색(뉴럴 검색-학습)+LLM 기반 생성
  - 검색기 평가
    - ./data/korquad_output/eval_questions_dev.json
    - 6,707개 질문 및 단락



Colab 화면



Train : closed test(학습에 사용한 질문으로 평가)
Dev : 블라인드 test(학습에 사용하지 않은 질문)

베이스모델 : 학습(finetuning) 하지 않은 모델
파인튜닝모델 : 학습한 모델

Copyright© 2021 by ETRI

- 한국어 문서 사실성 검증 기술
  - git clone https://github.com/ETRI-XAINLP/KorFactScore

Colab 화면

한국어 생성 문서의 단일사실관계를 검증하고 설명하는 기술
KorFactScore (Korean Factual precision in atomicity Score)

(한국어 생성 문서의 사실성 검증 및 한국어 언어모델 사실성 생성/판단 능력 평가)

Made with Python

Announcement

- 12/17/2024: "single_run" 브랜치(branch)를 새로이 생성하였습니다. 이 브랜치는 하나의 생성 문서만을 입력으로 받아서 처리하는 버전입니다. (c.f. 기존 버전은 여러개의 생성 문서들을 입력으로 받아서 한번에 배치(batch)로 처리하는 버전이었습니다.)

개발 배경

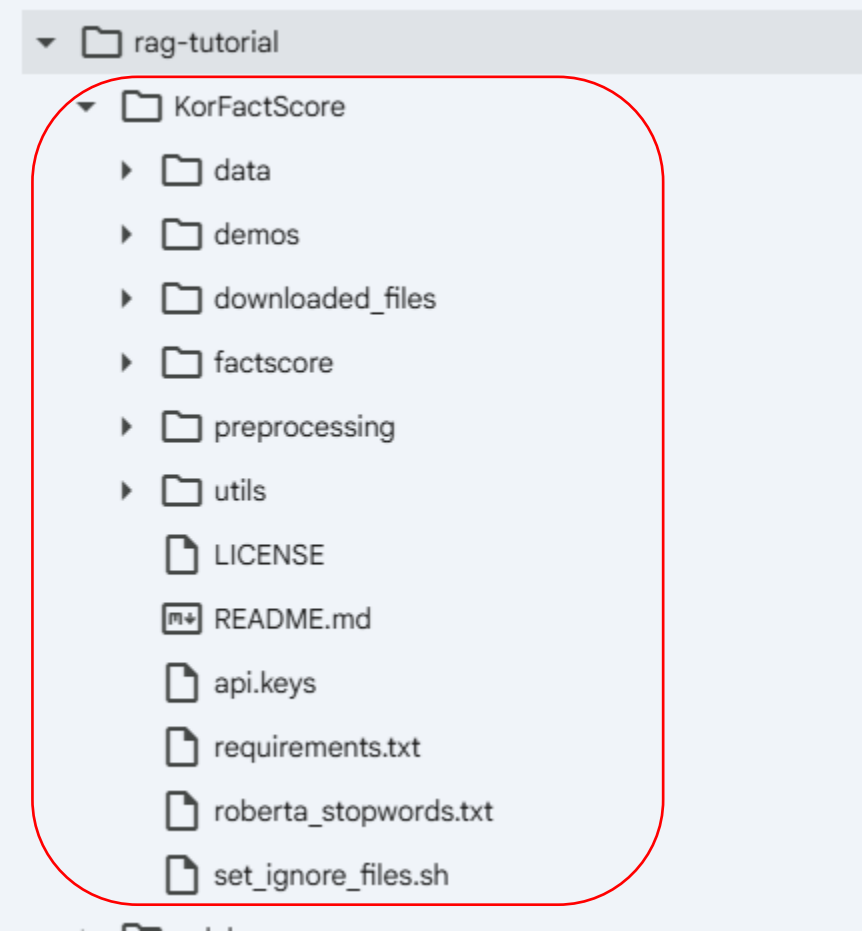
- 인공지능(AI)이 생성한 문서 내 정보가 신뢰할 수 있는 지에 대한 추가적인 검증이 필요함.
- 생성된 한국어 문서에 대해 개별 정보 단위로 사실 관계를 검증하고 설명할 수 있는 도구가 부재함.
- 특히, 최근 다양한 한국어 AI 언어 모델들로 대량의 문서들을 생성하고 있는 상황에서 이러한 기술의 필요성이 더욱 강조되고 있음.

주요 특징

- 한국어 생성 문서에 기술된 단일사실들(또는 원소사실, Atomic facts)에 대한 사실관계를 검증하고 설명하는 기술을 제공
- 특정 언어 모델에 제한되지 않으며(Model Agnostic), 평가용 언어모델을 교체하여 다양한 언어모델의 사실판단 능력을 평가할 수 있음
- 평가를 위한 데이터셋 제공: 생성된 문서, 단일 사실(atomic facts)로 분할된 문장들 포함
- 성능 평가를 위한 정답 데이터(ground truth label)와 평가 코드를 함께 제공

단일 사실(Atomic facts)

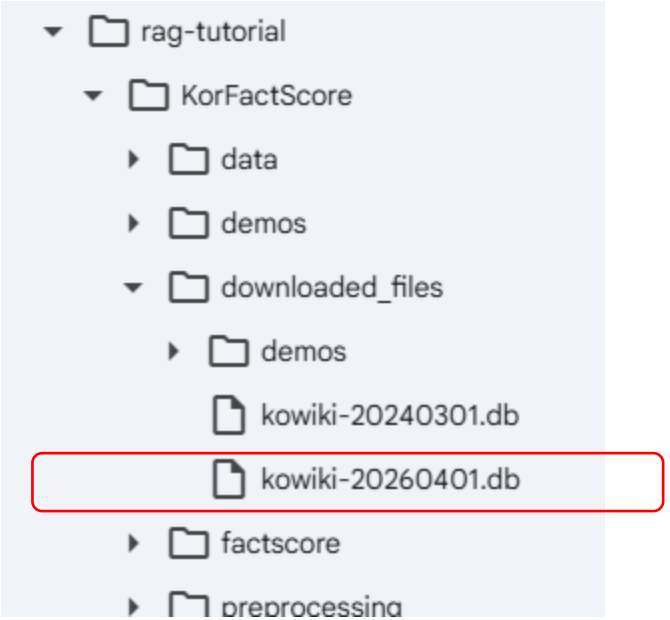
- 의미: 개별 의미를 담고 있는 작은 정보 단위
- 예시: 다음 문장은 3개의 단일 사실로 분해될 수 있음
  - "홍인한은 조선 후기의 문인이자 정치가이다." → ["홍인한은 조선 후기의 사람이다.", "홍인한은 문인이다.", "홍인한은 정치가이다."]



Copyright© 2021 by ETRI

- 한국어 문서 사실성 검증 기술
  - Wikipedia dump 다운로드한 후 sqlite 파일로 변환 필요
  - python build_kowiki_db.py (KorFactScore 상위 폴더에서 실행)

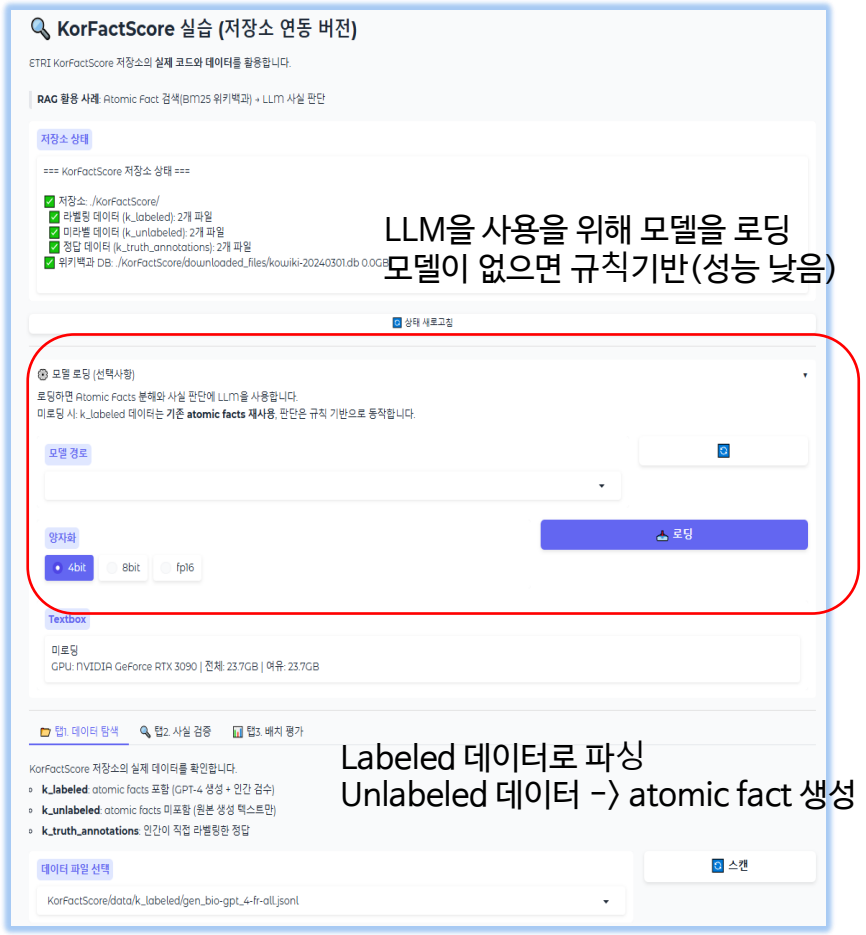
```
(rag_env_cu124) qa@qa:~/kmbae/aiacademy/cnu_lecture_26/KorFactScore$ python build_kowiki_db.py
[경고] mwparserfromhell 미설치 — 정규식 기반 파싱으로 대체합니다.
       pip install mwparserfromhell 을 실행하면 품질이 향상됩니다.

[INFO] 최신 완료 덤프 날짜: 20260401
[INFO] 덤프 날짜: 20260401
[다운로드] https://dumps.wikimedia.org/kowiki/20260401/kowiki-20260401-pages-articles.xml.bz2
  → ./wiki_dump/kowiki-20260401-pages-articles.xml.bz2
  100.0%  1.18/1.18 GB  2.4 MB/s
[완료] 1.18 GB 다운로드

[DB 생성 시작]
  입력 : ./wiki_dump/kowiki-20260401-pages-articles.xml.bz2
  출력 : ./KorFactScore/downloaded_files/kowiki-20260401.db

  [  10,000개 처리 |   10,000개 삽입 |      4개 건너뜀 | 574 doc/s]
  [  20,000개 처리 |   20,000개 삽입 |     11개 건너뜀 | 596 doc/s]
  [  30,000개 처리 |   30,000개 삽입 |     12개 건너뜀 | 605 doc/s]
  [  40,000개 처리 |   40,000개 삽입 |     18개 건너뜀 | 632 doc/s]
  [  50,000개 처리 |   50,000개 삽입 |     21개 건너뜀 | 649 doc/s]
  [  60,000개 처리 |   60,000개 삽입 |     26개 건너뜀 | 672 doc/s]
  [  70,000개 처리 |   70,000개 삽입 |     39개 건너뜀 | 690 doc/s]
```



Colab 화면

Copyright© 2021 by ETRI

- 한국어 문서 사실성 검증 기술
  - !python korfactscore_lab.py
  - 원하는 인덱스(표제어-사람)을 선택해서 진행
  - 인덱스 번호 선택 ➔ 레코드 보기 클릭 ➔ 사실 검증하기 버튼 ➔ 사실 검증 탭으로

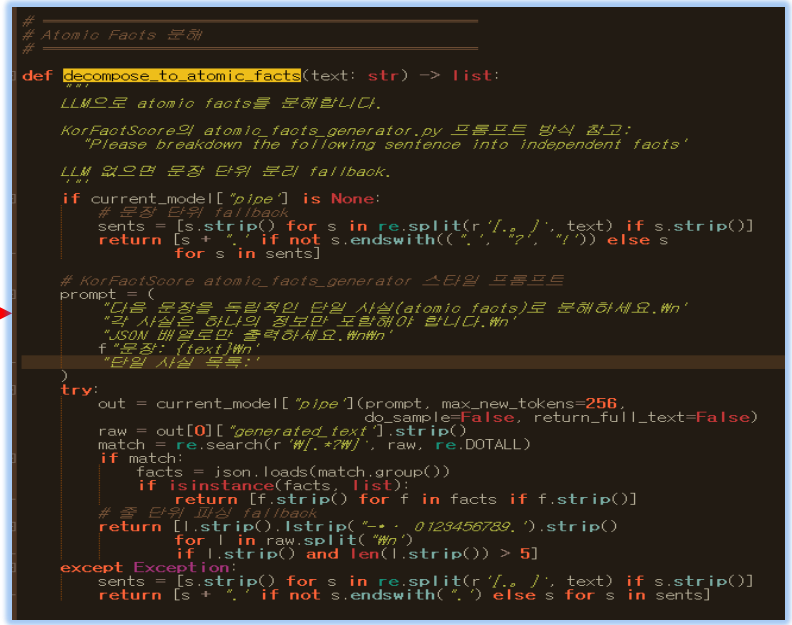


LLM을 사용을 위해 모델을 로딩
모델이 없으면 규칙기반(성능 낮음)

Labeled 데이터로 파싱
Unlabeled 데이터 -〉 atomic fact 생성

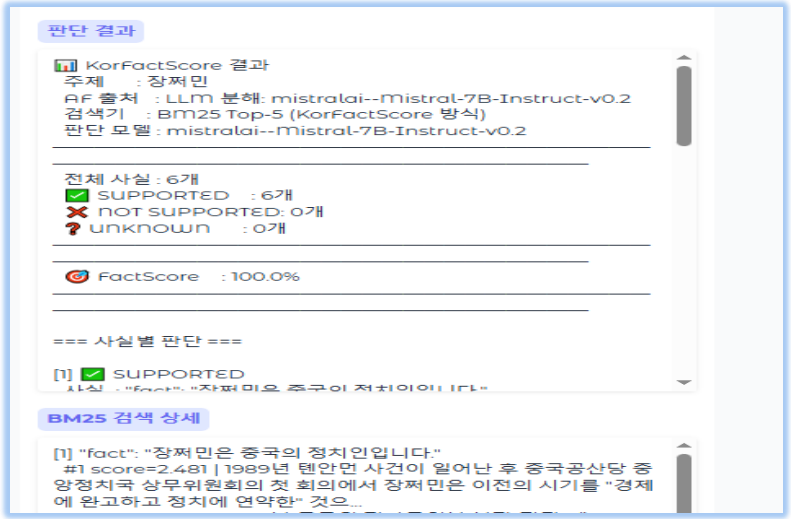
Copyright© 2021 by ETRI

- 한국어 문서 사실성 검증 기술
  - Unlabeled 데이터
  - atomic fact 생성



```
# ═══════════════════════════════════════
# Atomic Facts 분해
# ═══════════════════════════════════════

def decompose_to_atomic_facts(text: str) -> list:
    """
    LLM으로 atomic facts를 분해합니다.

    KorFactScore의 atomic_facts_generator.py 프롬프트 방식 참고:
      "Please breakdown the following sentence into independent facts"

    LLM 없으면 문장 단위 분리 fallback.
    """
    if current_model["pipe"] is None:
        # 문장 단위 fallback
        sents = [s.strip() for s in re.split(r'(?<=[.。!?])', text) if s.strip()]
        return [s + "." if not s.endswith(".", "?", "!") else s
                for s in sents]

    # KorFactScore atomic_facts_generator 스타일 프롬프트
    prompt = (
        "다음 문장을 독립적인 단일 사실(atomic facts)로 분해하세요.\n"
        "각 사실은 하나의 정보만 포함해야 합니다.\n"
        "JSON 배열로만 출력하세요.\n\n"
        f"문장: {text}\n"
        "단일 사실 목록:"
    )
    try:
        out = current_model["pipe"](prompt, max_new_tokens=256,
                                    do_sample=False, return_full_text=False)
        raw = out[0]["generated_text"].strip()
        match = re.search(r'\[.*?\]', raw, re.DOTALL)
        if match:
            facts = json.loads(match.group())
            if isinstance(facts, list):
                return [f.strip() for f in facts if f.strip()]
        # 줄 단위 파싱 fallback
        return [l.strip().lstrip("-• · 0123456789. ").strip()
                for l in raw.split("\n")
                if l.strip() and len(l.strip()) > 5]
    except Exception:
        sents = [s.strip() for s in re.split(r'(?<=[.。!?])', text) if s.strip()]
        return [s + "." if not s.endswith(".") else s for s in sents]
```

Atomic fact가 없으면 분리함. KorFactScore 만큼은 안됨



Copyright© 2021 by ETRI

- 한국어 문서 사실성 검증 기술

Copyright© 2021 by ETRI